DOG SCHEDULING APP
(PERSONAL PROJECT)

Design Report

Johnny Hui

March 3rd, 2026

Table of Contents

Project Overview 5
Problem Statement 5
Purpose 6
Milestones 6
Project Scope 8
System Overview 10
System Goals and Objectives 10
Primary Users of the System 12
Dog Owner 12
Dog Sitter 12
Minimum Viable Product (MVP) 12
System Architecture Design 13
Architecture Overview 13
Architecture Pattern 14
Technology Stack 15
Deployment Strategy 15
Data Design 17
Database Type 17
Core Entities Overview 17
Entity Relationship Diagram (ERD) 18
Entity Descriptions 20
Entity: User 20
Entity: DogSitter 21
Embedded Class: SitterLifestyle 23
Embedded Class: SitterPreferences 24
Entity: SitterService 25
Entity: Availability 26
Entity: Pet 27
Entity: PetTraits 29
Entity: PetRequest 30
Entity: Request 31
Entity: Booking 34

Entity: Review 36
Entity: Conversation 37
Entity: Message 38
User Interface Design (UI/UX) 39
User Stories 39
Dog Owner 39
Dog Sitter 40
User Flow Diagrams 41
Symbols and Definitions 41
User Flow (All Users) 42
User Flow (Dog Owner) 44
User Flow (Dog Sitter) 50
Wireframes 58
Splash Page 58
Login 59
All Users: Account Registration (Email & Password) 60
Dog Owner: Account Registration (Entering Profile Information) 61
Dog Owner: Account Registration (Uploading Profile Picture) 62
Dog Owner: Account Registration (Registering Pets) 63
Dog Sitter: Account Registration (Entering Sitter Information) 64
Dog Sitter: Account Registration (Entering Sitter Preferences) 65
Dog Sitter: Account Registration (Entering Sitter Lifestyle) 66
Dog Sitter: Account Registration (Entering Weekly Availability) 67
Dog Sitter: Account Registration (Setting Services Offered) 68
Finding Sitters 69
Viewing a Sitter's Profile Page 70
Dog Owner: Dashboard 71
Dog Sitter: Dashboard 71
Dog Owner: Creating a Booking Request 72
Dog Sitter: Receiving a Booking Request 73
All Users: Viewing a Booking 74
All Users: Canceling an Existing Booking 75
API Design 76
Overview 76
Endpoint Defintions 76
User Authentication Endpoints 76

User Endpoints (CRUD) 77
Pet Endpoints 77
Booking Request Endpoints 77
Booking Endpoints 78
Review Endpoints 78
DogSitter Endpoints 78
Message Endpoints 79
Security Design 80
Password Storage and User Authentication 80
Authorization Model 80
Token and Session Management 81
Data Protection 81
Change Log 82



PROJECT OVERVIEW

This project is simply a passion project aimed at simplifying the process of finding and scheduling dog-sitting services within a city.

The main deliverable is a platform consisting of a **web application** and an **iOS mobile application** that acts as an intermediary between **dog owners** and **dog sitters**, enabling users to easily discover, schedule, and book dog-sitting services.

PROBLEM STATEMENT

Many dog owners face difficulties when trying to find reliable and available dog-sitting services within their local area. Existing solutions are often fragmented, requiring owners to rely on personal networks, social media, or other platforms to locate trustworthy sitters.

This process can be very time-consuming and inconvenient to compare availability, pricing, services, and sitter reliability.

As a result, a centralized platform is needed to connect dog owners with dog sitters and streamline the booking and scheduling process.

PURPOSE

The purpose of this project is to simplify the process of finding, booking, and scheduling dog-sitting services. By creating an application that enables the discovery of dog-sitting services, the platform aims to reduce the frustrations and complexities experienced by both dog owners and dog sitters.

MILESTONES

Completion Date	Project Phase	Description
May 26th, 2026	Milestone I	**Project Design Report completed.**
June 15th, 2026	Milestone II	**Core back-end framework completed.** • Mission-critical entities (objects) created • Database schema defined • API endpoints defined • Controller and service layers
TBA	Milestone III	**Authentication & authorization implemented.** • User registration and login system • Password hashing (BCrypt) • Implement JWT generation and logic • Implement role-based access control for API endpoints
TBA	Milestone IV	**Implement Geo-search with Google Maps API** • Search based on address • Search based on radius/proximity • Test using a dummy front-end
TBA	Milestone V	**Database Integration** • Set up relational database (PostgreSQL) • Implement database access layer (DAO) • Ensure data consistency and relationships between entities
TBA	Milestone VI	**Front-end Implemented** • Build user interfaces for login, registration, and dashboards • Connect front-end to back-end APIs • Implement role-based interfaces (DogOwner vs. DogSitter views)

TBA	Milestone VII	**Testing and Validation** • Perform unit testing on backend services • Test front-end and back-end integration • Test authentication, authorization, and role-based API restrictions • Perform security validation (token expiry, tampering, invalid API access)
TBA	Milestone VIII	**Deployment & Reverse Proxy Setup** • Setup AWS services (EC2 instance) • Buy and register a domain name • Configure Nginx on EC2 instance • Configure Nginx to perform rate-limiting back-end API calls • Enable and test HTTPS-only connection using TLS certificates • Configure environment variables and production settings

PROJECT SCOPE

Scope Description (Features)	**IN SCOPE**	**OUT OF SCOPE**
	✓ User Creation System for different stakeholders ✓ Secure Login and User Authentication system (with JWTs) ✓ HTTPS Data Encryption (TLS certificates) ✓ Specific dashboards for different stakeholders ✓ Register dog(s) system to allow dog-sitters or businesses to view the dog-owner's associated pets (playful, fun, etc.) ✓ A Scheduling & Booking system (accept/reject) ✓ A system where dog-owners can view dog-sitter or business profiles (with ratings and a list of services) ✓ A system where dog-sitters or businesses can create a profile for dog-owners to discover (form-based) ✓ A rating system given for dog-sitters and dog-care businesses ✓ A search by city or current location (with filters) ✓ A functionality to view booking history ✓ A functionality to modify a booking	✘ Payment System ✘ Real-time GPS tracking system ✘ Veterinary service integration ✘ A flag/report/abuse system (for admins) ✘ Mobile version of the project

Acceptance Criteria	✓ The project will be deemed acceptable if it **passes all the test cases** and performs everything defined in the **system goals and objectives** section.
Project Deliverables	▪ A Web application ▪ An iOS mobile version of the web application ▪ A system where dog-owners can view dog-sitter or caring business profiles (with ratings and a list of services) ▪ A system where dog-owners can book an appointment with a dog-sitter or caring business. ▪ A functionality to view booking history ▪ A functionality to modify an existing booking/appointment
Constraints	▪ Limited team size (independent developer) ▪ Data security and privacy compliance (legal stuff) ▪ External cloud services can be costly (Google Maps API, S3, etc.) ▪ **Monolithic design** can cause multiple points of failure ▪ EC2 (free tier) limited system storage and performance
Stakeholders	▪ Dog owners ▪ Independent dog-sitters ▪ Doggie day-care business owners ▪ System Administrator / Developers
Assumptions	▪ Users have internet access ▪ Users have access to a PC or mobile device ▪ Users will only use this application for seeking dog services and not for dating or other purposes ▪ Users will regularly use the live messaging feature to connect with dog owners while appointments are active ▪ Sitters will update their availability regularly ▪ The application will initially serve one city

SYSTEM OVERVIEW

This section outlines the objectives and goals the system must achieve in order to meet the acceptance criteria, and the minimal viable product (MVP).

SYSTEM GOALS AND OBJECTIVES

The primary goal of the system is to provide a platform that enables dog owners to easily **discover, schedule,** and **book dog-care services** offered by both individual dog sitters and professional dog-care businesses. To achieve this goal, the system must be able to perform all of the tasks in the table below:

System Objectives	Description
Account Creation [Dog Owner]	The system must be able to provide dog-owners with a simple and easy-to-use signup interface for account creation.
Account Creation [Dog Sitter]	The system must be able to provide independent dog-sitters with a simple and easy-to-use signup interface for account creation.
Account Creation [Dog Care Business]	The system must be able to provide dog-care businesses with a simple and easy-to-use signup interface for account creation.
Securely Store and Present Photos from User Uploads	The system must be able to securely store and show user-uploaded profile photos and any photos of their pets to the front-end view.
Display Dog Caring Services Locally	The system must provide a feature for dog owners to search and discover available dog sitters and dog-care businesses within their area.
Live Messaging Between Dog Owner & Dog-sitter/Business	The system must be able to provide a live-messenger feature between a dog-owner and their associated dog-sitter or carer.
Book and Schedule [Dog Owner]	The system must be able to provide a simple and efficient scheduling interface for coordinating dog-care appointments.
Modify a Booking/Appointment [All Users]	The system must be able to allow dog-owners to modify a booking or appointment and have the change reflected to the affected dog-sitter(s) or business(s).
Accept/Reject a Booking [Dog-sitter/Business]	The system must be able to allow dog-sitters or care businesses with the feature to accept or reject a booking request made by dog-owners.

Rate a User [Dog-Owners Only]	The system must be able to allow dog-owners to rate a dog-sitter or caring business after a successful appointment.
View User Profiles [All Users]	The system must be able to display the user rating, the services offered, years of experience, and any pets the dog-sitter or dog-care business has.
View Booking History [All Users]	The system must be able to display a list of all previous bookings (history) made between a dog-owner and their dog-sitter.
Search Filter based on User Ratings [Dog Owners Only]	The system must be able to allow dog-owners to search for a dog-sitter or any dog-caring service based on user ratings.
Search Filter based on a Specific User [Dog Owners Only]	The system must be able to allow dog-owners to search for care services based on a specific user.
Search Filter based on a Specific Service [Dog Owners Only]	The system must be able to display care services for dog-owners depending on a specific service required.
Search Filter based on Availability [Dog Owners Only]	The system must be able to display care services for dog-owners based on dog-sitter's availability (date and time).
Modify & Manage User Profiles [All Users]	The system must allow dog sitters and businesses to create and manage profiles that describe their services offered, availability, and experience.
Provide Live Reminders for Upcoming Bookings	The system must be able to provide reminder notifications for upcoming bookings/appointments.
Data Storage	The system must maintain a reliable system for storing and managing user profiles, passwords and booking information and any photos.
Data Encryption	The system must be able to provide secure data transmission between a client and the server at all times.
User Authentication	The system must be able to provide stateless and secure user authentication during login and be able to differentiate the user's role (using JWTs).
Role-based API Authorization	The system must be able to perform role-based API access for specific users according to their roles.
Rate-Limiting API Calls	The system must be able to prevent API abuse by limiting the amount of API calls made available to specific users/devices.

Role Specific Interface	The system must be able to provide specific user interfaces according to the user role. Example: (dog owner, dog-sitter, dog-caring business, or admin)

PRIMARY USERS OF THE SYSTEM

During the initial phase, the system will cater to only two primary users of the system: **dog owners** and **dog sitters**. Later on, as the system becomes more defined, system administrators and dog-care businesses will be included.

Dog Owner

They are users who seek pet care services. They can create requests, book services, write reviews and communicate with DogSitters.

Dog Sitter

They are users who provide and offer pet care services. They can accept/reject requests for bookings, manage their own availabilities, and communicate with DogOwners.

MINIMUM VIABLE PRODUCT (MVP)

The **Minimum Viable Product (MVP)** represents the initial version of the system that provides the essential functionality required to connect dog owners with dog sitters. The MVP focuses on core features on a web application that allow users to securely create accounts, discover available dog sitters, schedule and maintain bookings. System security features such as data encryption, rate-limiting API calls, stateless user authentication using JWTs, role-based API authorization, and secure password storing (hash) will also be included.

The following features that will **NOT** be included in the MVP, but will be implemented in later versions of the system: live messaging, real-time booking notifications and reminders, a reporting or flagging system, advanced search functionality, and additional administrative features. Furthermore, the development of a refined user interface and an iOS mobile version of the application will be postponed until the core MVP features have been successfully implemented.

SYSTEM ARCHITECTURE DESIGN

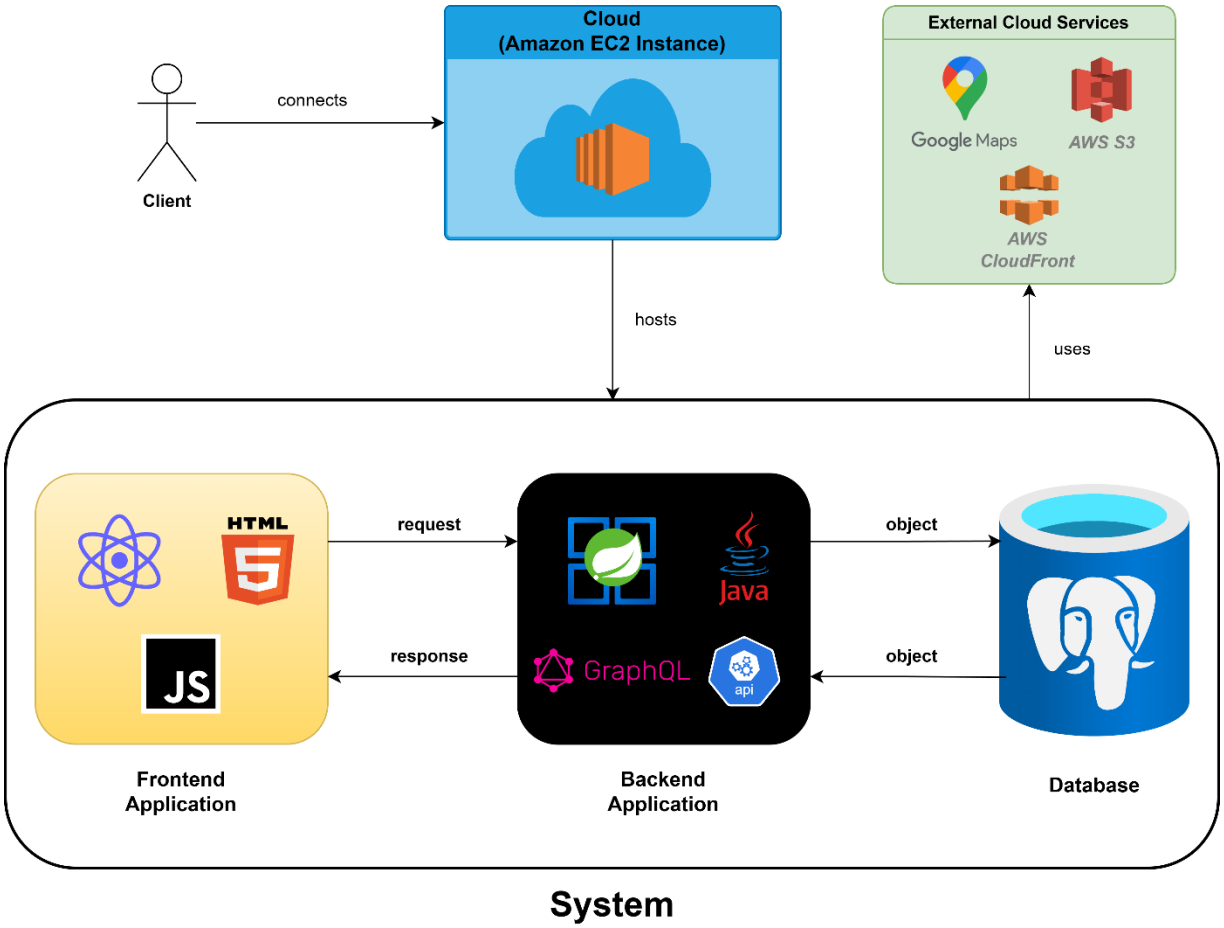
This section describes the overall system architecture, the architectural patterns used in the application, and the technology stack selected for the implementation. Figures and diagrams will be included to better visualize the architecture.

ARCHITECTURE OVERVIEW

The entire system follows a **monolithic**, **client-server architecture** where the frontend and backend are separated. The frontend (client) is implemented using **React**, which is responsible for rendering the user interface (UI) and interacting with the backend (server) through RESTful API calls.

The backend will be developed and managed using the **Spring Framework**. The backend will expose RESTful API endpoints that allow the React frontend client to communicate with the system. It will handle incoming requests, process and validate data, execute business logic, manage user authentication and authorization, and handle database operations.

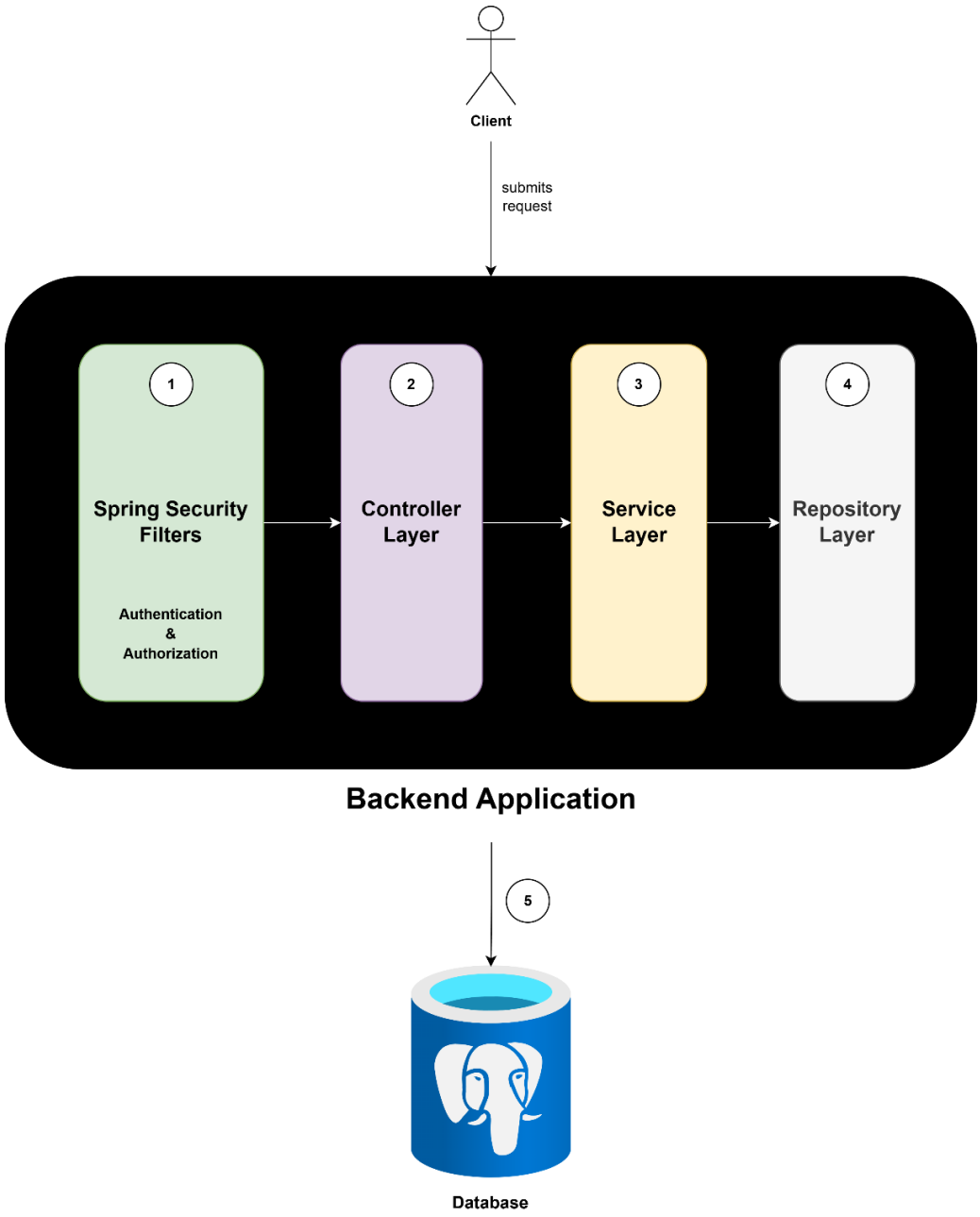
Lastly, the system will use **PostgreSQL** as its relational database management system for persistent data storage. The entire system and workflow will be **object-oriented.**



ARCHITECTURE PATTERN

The backend system will make use of the **layered architecture** using the **Spring Framework** which consists of several security filters (JWT & role-based authentication), controller, service, repository (data access), and the database layers. This separation improves maintainability, scalability, and testability of the application.

The controller layer handles specific HTTP API requests from the client and delegates the request to a specific service. The service layer will perform validation and apply business logic and calls upon the repository (data access layer) to perform any database operations. The data sent from the backend to the frontend (view) will be of JSON format.



TECHNOLOGY STACK

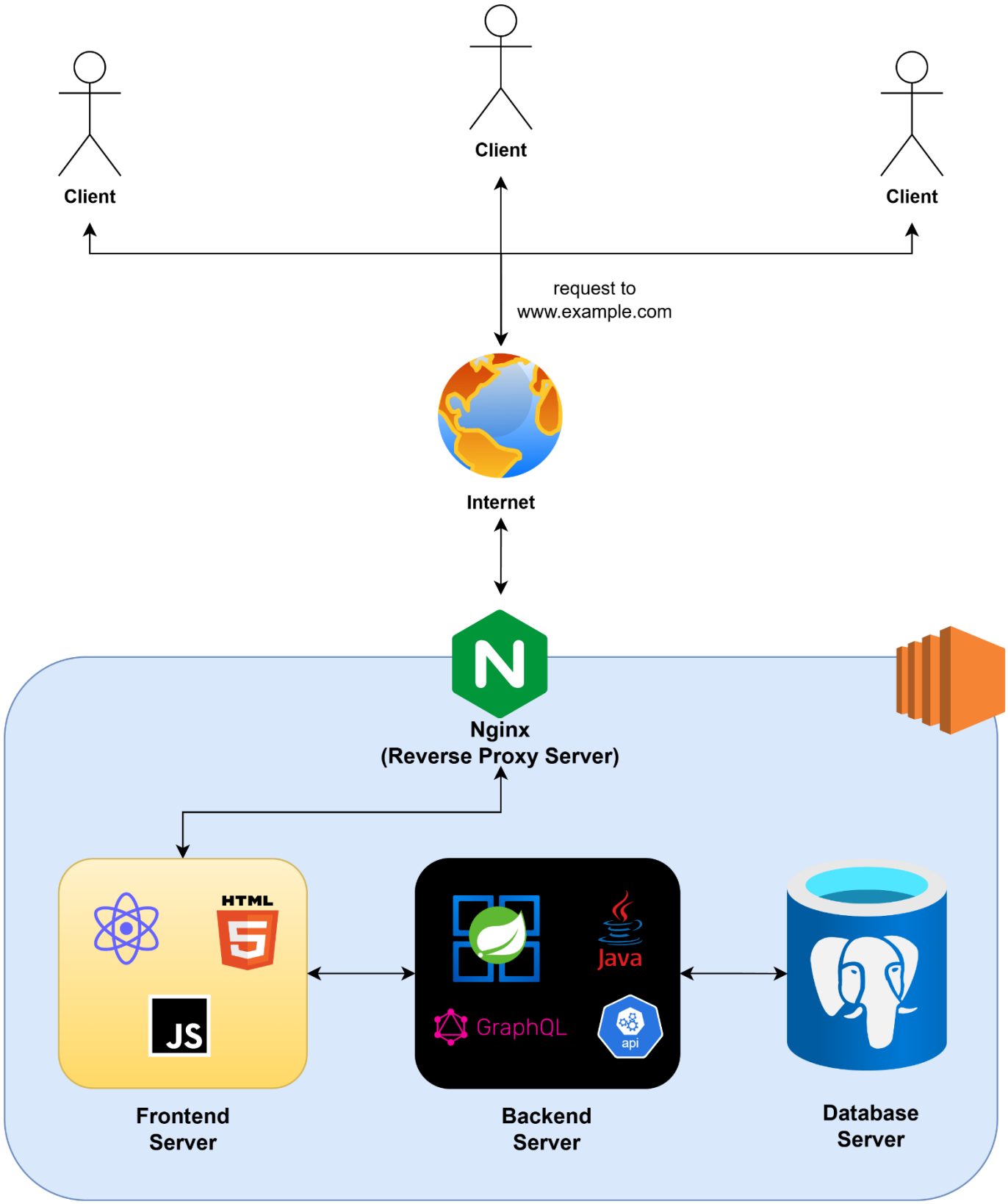
Components	Technologies Used				
Frontend	React	JavaScript	HTML	CSS	Material UI
Backend	Spring Boot & JPA/Hibernate	Spring Security	Java	GraphQL	JWT
Authentication	Spring Security	JWT	ECDSA signatures		
Authorization	Role-based	Spring Security			
Cloud Deployment	Amazon EC2	NGINX (Reverse Proxy)			
Client/Server Encryption	HTTPS TLS/SSL	NGINX (Reverse Proxy)			
Photo Storage and Delivery	Amazon S3	Amazon CloudFront			
Real-time Live Messenger	Spring WebSockets				
Geographic Map	Google Maps API				

DEPLOYMENT STRATEGY

The entire system will follow a **monolithic architecture**, where the application will be deployed and executed as a single unit on an **Amazon EC2 instance**. Within this environment, the front-end application, back-end application, and database will run concurrently on the same instance.

A **domain name** will be purchased and used to easily allow clients to access the EC2 instance web application using just the domain name. A **reverse proxy server** will be configured in the EC2 instance to directly route and forward initial client requests directly to internal application servers.

This deployment strategy is current go-to solution, as it is cost-effective and simplifies deployment and system management by combining everything into a single environment. A diagram of this strategy is shown below.



EC2 Cloud Instance
(Application Server)

DATA DESIGN

DATABASE TYPE

The system will use a relational database management system (RDBMS), specifically **PostgreSQL**. A relational model was chosen due to its strong support for structured data, relationships between entities, and transactional consistency. Additionally, this database model goes hand-in-hand with **Spring Boot** and **JPA/Hibernate** frameworks; thus - will make database CRUD operations, queries and development much easier.

CORE ENTITIES OVERVIEW

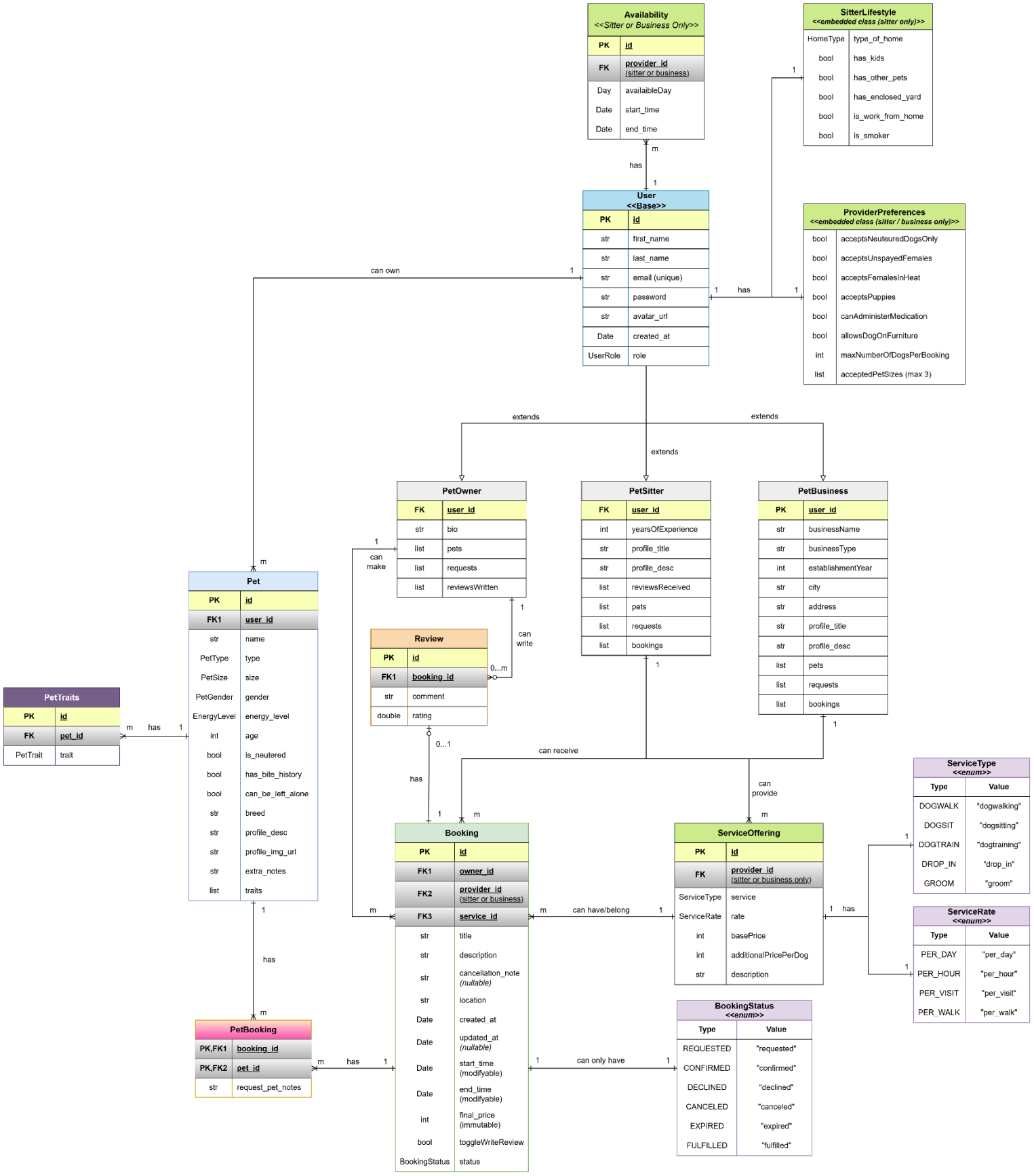
The system is designed to streamline the process of helping dog owners easily discover and connect with nearby dog sitters. Once a suitable dog sitter is identified, the owner can submit a service request for a specific date, time slot, and type of service.

After the dog sitter reviews the request, it is either accepted or declined. If accepted, the request transitions into a booking, representing a confirmed agreement between the dog owner and the dog sitter. While a booking reflects a finalized arrangement, certain fields may remain editable to accommodate minor adjustments such as time start/end changes.

The table below outlines the core entities within the database and explains how each contributes to the overall business workflow of the system.

Entity	Description
User	Stores basic user information. Users can register Pet(s) for servicing and send requests for an appointment (booking).
DogSitter	Stores DogSitter details. DogSitters can review requests, set their availability & customize their services.
SitterService	Defines services offered by DogSitters.
Request	A link between a DogOwner and a DogSitter. Includes all the information necessary for a dogsitter to understand the service or booking requested.
Pet	Stores essential pet details, allowing DogSitters to understand the dog's needs and care requirements.
Booking	Represents an accepted request between a DogSitter and a DogOwner for a specific dog service.

ENTITY RELATIONSHIP DIAGRAM (ERD)



ENTITY DESCRIPTIONS

This section provides a brief overview of each entity in the system, including its purpose, attributes, constraints, and the relationships between them; Enums will not be covered.

Entity: User

Description

The User entity represents a primary user of the system who seeks services for their dogs. They serve as a central component of the platform, interacting and connecting with other entities.

Attributes

Attribute	Type	Description	Constraints
id	Long	Used as a primary key	Primary Key
first_name	String	The first name of the user	Not Null
last_name	String	The last name of the user	Not Null
email	String	The email for the associated user (used for login)	Not Null, Unique
password	String	Plain-text password (saved as a hash)	Not Null
postal_code	String	The postal code of the user (used for Google Maps API)	Not Null
profile_desc	String	A brief description of the DogOwner	N/A
profile_img_url	String	Used to load user uploaded images	Default (stock photo used)
pets	List<Pet>	Stores a list of pets for the DogOwner	N/A
requests	List<Request>	Stores a list of Requests for the DogOwner	N/A
bookings	List<Booking>	Stores a list of Bookings for the DogOwner	N/A
role	UserRole <Enum>	Used to distinguish user roles	Not Null

Relationships

- A **DogOwner** can register and have 1-to-Many **Pets** (1:M)
- A **DogOwner** can have 0-to-Many Conversations with other **DogSitters** (0:M)
- A **DogOwner** can make, modify, or cancel 0-to-Many **Requests** (0:M)
- A **DogOwner** can make and have 0-to-Many **Bookings** (0:M)
- A **DogOwner** can make 0-to-Many **Reviews** from successful Bookings (0:M)

Constraints & Business Rules

- Email field must be unique.
- Password field must be hashed.
- A DogOwner must register at least 1 Pet to interact with the system.
- A DogOwner may face additional costs per Pet (depends on the DogSitter).

Entity: DogSitter

Description

The DogSitter entity represents service providers who offer pet care services within the system. Their primary role is to advertise available services, enabling DogOwners (users) to discover suitable sitters and initiate bookings for their pet(s). DogSitters can manage and customize their profiles, define their availability, and specify the range of services they offer.

Attributes

Attribute	Type	Description	Constraints
user_id	Long	Used as a primary key (from id of the User entity)	Primary Key, Foreign Key
profile_title	String	The bio title for the Dogsitter's profile	N/A
years_of_experience	Integer	Represents the years of sitting experience	N/A
avg_rating	Double	An overall average of the ratings given by DogOwners (based off of reviews)	N/A
last_active	Date	The date when the DogSitter was last active	N/A
availabilities	List<Availability>	Stores a list of available dates for service	N/A
servicesOffered	List<SitterService>	Stores a list of Services of the Dogsitter	N/A
reviews	List<Reviews>	Stores a list of Reviews for the DogSitter	N/A

Relationships

- A **DogSitter** can register and have 1-to-Many **Pets** (1:M)
- A **DogSitter** can have 0-to-Many Conversations with **Users** (0:M)
- A **DogSitter** can customize and offer 1-to-Many **SitterServices** (1:M)
- A **DogSitter** can receive, modify, or cancel 0-to-Many **Reviews** (0:M)
- A **DogSitter** can modify and have 0-to-Many **Bookings** (0:M)
- A **DogSitter** can customize and have only 1 **SitterLifestyle** (1:1)
- A **DogSitter** can customize and have only 1 **SitterPreference** (1:1)

Constraints & Business Rules

- [General Requirements]
  - A **DogSitter** must complete a questionnaire that answers most fields in the **SitterLifestyle** and **SitterPreferences** during account creation.
  - A **DogSitter** must have at least one date available for **Availability**.
  - A **DogSitter** must be able to offer at least one **SitterService**.
  - A **DogSitter** must have an official and valid postal code in order to pass account creation.

- [Fields]
  - Email field must be unique.
  - Password field must be hashed.
  - The system must be able to save and record the date last active for the DogSitter's account.
  - The system must be able to require the DogSitter to update their availability once every month.
  - If no *profile_img_url* is provided, then a default stock profile image will be used.

Notes

- A **DogSitter** inherits from the User class (entity).
- A **DogSitter** can perform everything a User can do, but with special caveats described above.

Embedded Class: SitterLifestyle

Description

This class represents aspects of a DogSitter's lifestyle, and a glimpse of their everyday life. This class will contribute to a more detailed sitter profile and therefore - will help DogOwners make more informed decisions when searching for suitable sitters.

Attributes

Attribute	Type	Description	Constraints
type_of_home	HomeType (Enum)	The Dogsitter lives in a house or apartment	Not Null
has_kids	bool	Determines if DogSitter has any kids	Not Null
has_other_pets	bool	Determines if DogSitter has other pets	Not Null
has_enclosed_yard	bool	Determines if DogSitter has an enclosed yard	Not Null
is_work_from_home	bool	Determines if DogSitter is working from home	Not Null
is_smoker	bool	Determines if DogSitter is a smoking individual	Not Null

Relationships

- A **DogSitter** has one-and-only one **SitterLifestyle** and can customize it to their liking (1:1)

Constraints & Business Rules

- The **DogSitter** must complete and fill out all fields within this class during account creation.
- The system must provide a form for the DogSitter to fill out in order to fulfill this class.
- The system must be able to allow the DogSitter to modify any of these fields at any time.

Notes

- This class is **NOT** a standalone entity, but rather an embedded component within the **DogSitter** entity that is used to encapsulate and group a set of related attributes.

Embedded Class: SitterPreferences

Description

This class represents a DogSitter's service preferences, defining the conditions and boundaries under which they provide care. It allows DogOwners to clearly understand the sitter's expectations and limitations when considering them for their pet's needs.

Attributes

Attribute	Type	Description	Constraints
acceptsNeuteredDogsOnly	bool	Determines if the DogSitter accepts only neutered dogs	Not Null
acceptsUnspayedFemales	bool	Determines if the DogSitter accepts unspayed female dogs	Not Null
acceptsFemalesInHeat	bool	Determines if the DogSitter allows sitting for female dogs in heat	Not Null
acceptsPuppies	bool	Determines if the DogSitter accepts servicing of puppies	Not Null
canAdministerMedication	bool	Determines if the DogSitter can administer any medications for pets	Not Null
allowsDogOnFurniture	bool	Determines if the DogSitter allows dogs on the furniture	Not Null
maxNumberOfDogsPerBooking	int	Determines the maximum number of dogs a DogSitter is willing to accept per booking	Not Null
acceptedPetSizes	List<PetSize>	Determines the pet size(s) that the sitter is willing to accept	Not Null, Max 3

Relationships

- A **DogSitter** has One-and-Only One **SitterPreferences** and can customize it to their liking (1:1)

Constraints & Business Rules

- The **DogSitter** must complete and fill out all fields within this class during account creation.
- The system must provide a form for the DogSitter to fill out in order to fulfill this class.
- The system must be able to allow the DogSitter to modify any of these fields at any time.
- *maxNumberOfDogsPerBooking* must be applied during a DogOwner's booking request.

Notes

- This class is **NOT** a standalone entity, but rather an embedded component within the **DogSitter** entity that is used to encapsulate and group a set of related attributes.

Entity: SitterService

Description

This entity represents all the services offered by any one DogSitter. Each record (row) will contain service information such as the type of service, the rate charged and the base price. This will allow DogOwners to determine if a specific DogSitter has the service they require for their pet and whether it is within their budget.

Attributes

Attribute	Type	Description	Constraints
id	Long	Used as a primary key	Primary Key
sitter_id	Long	Used as a foreign key to join DogSitter entity (table)	Not Null
service	ServiceType (enum)	Determines the type of service provided by the DogSitter. Saved as a string (varchar).	Not Null
rate	ServiceRate (enum)	Determines the rate charged for a particular service. Saved as a string (varchar).	Not Null
basePrice	int	Defines the base price charged for a particular service	Not Null, Non-Negative
additionalPricePerDog	int	Defines the additional price per dog/pet a particular service	Non-Negative
description	int	A brief description for the service	Not Null, Max (100)

Relationships

- A **DogSitter** can offer 1-to-Many **SitterServices** and can customize it to their liking (1:M)

Constraints & Business Rules

- The **DogSitter** must complete and fill out all fields within this class during account creation.
- The system must provide a form for the DogSitter to fill out in order to fulfill this class.
- *BasePrice* and *AdditionalPricePerDog* cannot hold a negative value.
- *ServiceRate* must hold the following values: per day, per hour, per visit, or per walk
- *ServiceType* must hold the following values: dog walking, dog sitting, dog training, drop in, or grooming

Entity: Availability

Description

This entity represents a DogSitter's weekly availability. Each record (row) corresponds to a specific day of the week and defines the time slot(s) during which the sitter is available on that day.

Attributes

Attribute	Type	Description	Constraints
id	Long	Used as a primary key	Primary Key
sitter_id	Long	Used as a foreign key to join DogSitter entity (table)	Not Null
day_of_the_week	Date	Represents a specific day of the week	Not Null
start_time	Date	The allocated start time for the day	Not Null
end_time	Date	The allocated end time for the day	Not Null
last_updated_at	Date	The last updated time the DogSitter has updated their availability	N/A

Relationships

- A **DogSitter** can have One-to-Many days and timeslots for their **Availability** (1:M)

Constraints & Business Rules

- All **DogSitters** must have at least one available day and is able to work for at least one hour.
- All **DogSitters** must be required to update their **Availability** once every month.
- The **DogSitter's** availability must remain consistent with their confirmed **Bookings**.
  - Time slots that overlap with existing bookings are automatically marked as unavailable, preventing other **DogOwners** from selecting the same period.
- When selecting an available day, the minimum available hour must be at least 1 hour.
- The *start_time* must be before *end_time*.
- The system must restrict *day_of_the_week* to actual days of the week (M, T, W, TH, F, SAT, SUN)
- The system must show all of the sitter's availabilities when a DogOwner is creating a **Request** for servicing.
- The system must be able to provide a form in the view for DogSitters to fill out their availability.

Entity: Pet

Description

This entity represents a DogOwner's **pet** and contains relevant information such as age, breed, behavioral traits, and size. By providing comprehensive and structured information, this entity enables DogSitters to assess whether they can safely and appropriately accommodate the pet before accepting a request for service.

Attributes

Attribute	Type	Description	Constraints
id	Long	Used as a primary key	Primary Key
user_id	Long	Used as a foreign key to join the User entity	Nullable
name	String	Defines the name of the pet	Not Null, Max (20)
type	PetType (Enum)	Saved as a string (varchar)	Not Null
size	PetSize (Enum)	Saved as a string (varchar)	Not Null
energy_level	EnergyLevel (Enum)	Saved as a string (varchar)	Not Null
age	Integer	Defines how old the Pet is	Not Null, Non-Negative
is_neutered	bool	Defines whether the Pet is neutered	Not Null
has_bite_history	bool	Defines whether the Pet has a bite history	Not Null
can_be_left_alone	bool	Defines whether the Pet can be left alone	Not Null
breed	String	Defines the breed of the Pet	Not Null, Max (50)
profile_desc	String	A brief description for the Pet's profile	Not Null, Max (100)
profile_img_url	String	The URL link for the Pet's profile picture	Default (Stock Photo)
extra_notes	String	Special handling notes (optional)	Nullalble, Max (100)
traits	List<PetTrait>	Defines the behavioral traits of the Pet	Not Null, Min (3)

Relationships

- A **Pet** exclusively belongs to only One **DogOwner** (M:1)
- A **Pet** exclusively belongs to only One **DogSitter** (M:1)
- A **Pet** can belong to One-to-Many **PetRequests** (1:M)
- A **Pet** can have One-to-Many **PetTraits** (1:M)

Constraints & Business Rules

- [General Requirements]
  - All **DogOwners** must own and register at least one pet in order to use the system.
  - **DogSitters** are not required to register a pet in order to use the system.
  - A **Pet** must have at least three traits in order for it to be registered.
  - At least one **Pet** must be included per **Request** for servicing with any DogSitter.
  - The system must provide a form for DogOwners and DogSitters to register their pet.

- [Fields]
  - *Name* field cannot exceed 20 characters.
  - *Breed* field cannot exceed 50 characters.
  - *Profile_desc* field cannot exceed 100 characters.
  - *Extra_notes* field cannot exceed 100 characters.
  - *Age* field cannot hold a negative value.

Notes

- **PetRequests** is a bridging table that links **Pets** to **Requests**.

Entity: PetTraits

Description

This entity allows **Pets** to have multiple traits. These traits allow DogSitters to better understand an owner's Pet before accepting them in for a Booking.

Attributes

Attribute	Type	Description	Constraints
id	Long	Used as a Primary Key	Primary Key
pet_id	Long	The unique id of a Pet object	Foreign Key, Not Null
trait	PetTrait <Enum>	A single trait for a Pet. Saved as a string (varchar)	Not Null

Relationships

- A single **PetTrait** holds information for a single **Pet** from the foreign key (M:1)
- A single **PetTrait** includes one and only one trait from the PetTrait enum (1:1)

Constraints & Business Rules

- The system must be able to create a new row for each PetTrait for a single Pet.
- The *pet_id* field is a foreign key and must NOT be null.
- The *trait* field must NOT be null.

Entity: PetRequest

Description

This entity is simply a bridging entity between a **Request** and a **Pet** entity to address a Many-to-Many relationship. For this system, a Request can include many Pets. On the other hand, any single Pet can belong to many Requests overtime. In this fashion, the queries for this entity allows the system to get all Pets belonging to a single Request or get all Requests belonging to a single Pet.

Attributes

Attribute	Type	Description	Constraints
request_id	Long	The unique id of a Request object	Primary Key <Composite>
pet_id	Long	The unique id of a Pet object	Primary Key <Composite>
request_pet_notes	String	An optional string containing any special pet instructions provided by a DogOwner	Nullable, Max (100)

Relationships

- A **PetRequest** holds information for a single **Request** (M:1)
- A **PetRequest** holds information for a single **Pet** (M:1)

Constraints & Business Rules

- The system must be able to create a new row for each Pet included for a single Request.
- *request_id* and *pet_id* are foreign keys and must NOT be null.
- *request_pet_notes* cannot exceed 100 characters.

Entity: Request

Description

This entity represents an initial booking request submitted by a DogOwner for a specific service offered by a DogSitter. It captures all relevant details needed for the DogSitter to evaluate the request, including the requested service, selected date and time, and the pets involved.

Attributes

Attribute	Type	Description	Constraints
id	Long	Used as a Primary Key	Primary Key
owner_id	Long	The unique id of the DogOwner	Foreign Key <Not Null>
sitter_id	Long	The unique id of the DogSitter	Foreign Key <Not Null>
title	String	The title or subject of the Request	Nullable, Max (100)
description	String	A brief description for the service requested	Nullable, Max (200)
extra_notes	String	DogOwners can include extra notes in this field	Nullable, Max (200)
cancellation_notes	String	DogOwners or DogSitters can leave notes for cancellation of the request	Nullable, Max (200)
location	String	The address of the requested service to be performed	Not Null
created_at	Date	The creation date of the Request	Not Null
expiry_date	Date	A dynamic value set by the system according to the date of the Request created and the *start_time* for the Booking.	Not Null
start_time	Date	The start date and time of the requested service	Not Null
end_time	Date	The end date and time of the requested service	Not Null
time_modified	Date	The date and time that the request has been modified	Nullable
is_modified	Bool	Checks whether the request has been modified	Default (false)
shouldMeetAndGreet	Bool	Checks whether a meet & greet should be performed before Request is accepted	Default (false)
meetAndGreetTime	Date	The date and time for a meet & greet	Nullable
price	Integer	The price of the service to be performed	Not Null, Non-Negative
service_type	ServiceType <Enum>	The type of service to be performed. Saved as a string (varchar)	Not Null
status	RequestStatus <Enum>	The status of the Request. Saved as a string (varchar)	Default (PENDING)

Relationships

- A **Request** holds information for a single **DogOwner** from Foreign Key (M:1)
- A **Request** holds information for a single **DogSitter** from Foreign Key (M:1)
- A **Request** becomes a **Booking** if it is accepted (1:1)
- A **Request** has One-to-Many **PetRequests** (1:M)

Constraints & Business Rules

- [General Requirements]
  - A **Request** must include at least one **Pet**.
  - A **Request** for service can only be submitted 3 hours in advance of current time.
  - A **Request** by default must hold the *RequestStatus* as "PENDING".
  - A **Request** must not be deleted from the database.
  - Shall a Request be "ACCEPTED", the system must create a **Booking**.

- [Service Time]
  - The start time must NOT exceed the end time.
  - [Time Validation] - The system must validate whether the selected start and end times do not conflict with any of the DogSitter's booked times.
  - [Per/Hour service] - The service time requested must be at least 1 hour or greater.
  - [Per/Visit service] - The service time requested must be at least 1 hour or greater.
  - [Per/Walk service] - The service time requested must be at least 1 hour or greater.
  - [Per/Day service] - The service time requested must be at least 1 day or greater.
  - Services with a ServiceRate of "PER_VISIT" or "PER_WALK" have a fixed (flat) price, and therefore - support assignable service times that operate on an hourly basis.

- [Expiry Time]
  - The system must change the RequestStatus to "EXPIRED" should the current server date time exceeds the *end_time*.

  - The system must dynamically determine the request expiry time based on the request's creation timestamp and the scheduled Booking start time:
    - If the booking is greater than 3 days away, then the expiry is 48 hours
    - If the booking is either 2 or 3 days away, then the expiry is 36 hours
    - If the booking is one day away, then the expiry is 12 hours
    - If the booking is on the same day, then the expiry is 3 hours before start time
    - If the booking is on the same day and the start time is exactly 3 hours from the current time, the request expiry shall be set to 1 hour.

- [Modification]
  - The system must update the *is_modified* field shall the Request be modified.
  - The system should allow the start and end times be changed.
  - The system must validate the DogSitter's availability whenever a date change is requested, ensuring that the new time does not conflict with any of their existing bookings.
  - The system should allow DogOwners to change the number of Pets initially included.
  - The system should allow DogSitters and DogOwners to modify the Request.

- [Cancellation]
  - The cancellation of a Request can be performed by either DogOwners or DogSitters.
  - A *cancellation_note* must be included shall a Request be cancelled.
  - The system must change the request status to "REJECTED" shall a Request be cancelled.
  - Once a Request has been cancelled, the system must prevent it from being modified any further.

- [Fields]
  - The *price* field cannot hold a negative value and cannot be changed.
  - The *price* field is automatically tied to the ServiceType and ServiceRate, which is individually set by the DogSitter.
  - Title field cannot exceed 100 characters.
  - Description field cannot exceed 200 characters.
  - extra_notes and cancellation_notes cannot exceed 200 characters.

Notes

- [Meet & Greet]
  - This is an optional field and just includes information for the date and time of the meeting.
  - This functionality is independent and not managed by the system.
  - In other words, the decision to arrange and successfully complete a meet-and-greet rests entirely with the DogOwner and DogSitter.
  - The messaging system will help facilitate this process for communication.

Entity: Booking

Description

This entity represents an accepted Request and contains finalized booking details, including the service to be performed, the location, the date and time. While it closely resembles a Request, the Booking differs in that it signifies a confirmed, mutual agreement between the DogOwner (and their pets) and the DogSitter. Most of these fields are ported over from the Request entity once accepted.

Attributes

Attribute	Type	Description	Constraints
id	Long	Used as a Primary Key	Primary Key
request_id	Long	The unique id of the associated Request	Foreign Key <Not Null>
owner_id	Long	The unique id of the DogOwner	Foreign Key <Not Null>
sitter_id	Long	The unique id of the DogSitter	Foreign Key <Not Null>
title	String	The title or subject of the Booking	Not Null
description	String	A brief description for the service requested	Not Null
cancellation_note	String	DogOwners or DogSitters can leave notes for cancellation of the Booking (Optional)	Nullable
location	String	The address of the requested service to be performed	Not Null
created_at	Date	The creation date of the Booking	Not Null
updated_at	Date	The date and time that the Booking has been modified	Nullable
start_time	Date	The start date and time of the Booking	Not Null
end_time	Date	The end date and time of the Booking	Not Null
final_price	Integer	The final price of the service to be performed	Not Null
toggleWriteReview	Boolean	Determines if DogOwner wants to write a Review for a sucessful Booking	Default (True)
service_type	ServiceType <Enum>	The type of service to be performed. Saved as a string (varchar)	Not Null
status	BookingStatus <Enum>	The status of the Booking. Saved as a string (varchar)	Default (PENDING)

Relationships

- Zero-to-Many **Bookings** can belong to one **DogOwner** (0:M)
- Zero-to-Many **Bookings** can belong to one **DogSitter** (0:M)
- A **Booking** derives from a **Request** once it has been accepted (1:1)
- A fulfilled **Booking** can become associated with one **Review** (1:1)

Constraints & Business Rules

- [General Requirements]
  - The system must be able to create a Booking once a Request has been accepted.
  - The system must be able to port over most fields from the Request to the Booking.
  - The system (by default) must set the status to "PENDING" upon creation of the Booking.
  - A Booking must NOT be deleted from the database.

- [Modification]
  - The system should allow DogSitters and DogOwners to modify the Booking.
  - The system must be able to provide a form for modification of the Booking.
  - The system must provide a date for the *updated_*at field shall the Booking be modified.
  - The system should only allow the start and end times be changed.
  - The system must validate the DogSitter's availability whenever a date change is requested, ensuring that the new time does not conflict with any of their existing bookings.

- [Cancellation]
  - The cancellation of a Booking can be performed by either DogOwners or DogSitters.
  - A *cancellation_note* must be included shall a Booking be cancelled.
  - The system must change the Booking status to "CANCELLED" shall a Booking be cancelled.
  - Once a Booking has been cancelled, the system must prevent it from being modified any further.

- [Corresponding Reviews]
  - *toggleWriteReview* determines if the DogOwner wants to write a Review for a successful Booking.
  - Initially, this value is True and should only prompt the user when the Booking date has passed and the BookingStatus is FULFILLED.

Notes

- [BookingStatus]
  - FULFILLED means that the DogOwner has confirmed the Booking was successful
  - AWAITING means that the Booking has yet to be confirmed successful or failed
    - Status changes from PENDING after scheduled booking time has passed
  - PENDING means that the Booking is scheduled and has not yet to be performed
  - CANCELLED means that the Booking was manually cancelled or has been deemed unsuccessful by the DogOwner.

Entity: Review

Description

This entity represents a Review that contributes to the credibility of DogSitters. Reviews are generated from successfully fulfilled Bookings and may be optionally submitted by DogOwners.

Attributes

Attribute	Type	Description	Constraints
id	Long	Used as a Primary Key	Primary Key
booking_id	Long	The unique id of the associated Booking	Foreign Key <Not Null>
owner_id	Long	The unique id of the DogOwner (author)	Foreign Key <Not Null>
sitter_id	Long	The unique id of the associated DogSitter	Foreign Key <Not Null>
author	String	The first and last name of the DogOwner (author)	Not Null
service_type	String	The type of service performed for the Booking	Not Null
created_at	Date	The date of the Review created	Not Null
rating	BigDecimal	A rating from 0.0 – 5.0	Not Null, Check (0.0 and 5.0)

Relationships

- A **DogOwner** can write Zero-to-Many **Reviews** from successful Bookings (0:M)
- A **DogSitter** can have Zero-to-Many **Reviews** from successful Bookings (0:M)
- A **Review** holds information for the associated Booking from Foreign Key (1:1)
- A **Review** holds information for the corresponding author from Foreign Key (M:1)
- A **Review** holds information for the corresponding DogSitter from Foreign Key (M:1)

Constraints & Business Rules

- [General Requirements]
  - The Review can optionally come directly from an associated & successful Booking.
  - *booking_id*, *owner_id*, and *sitter_id* are foreign keys and must NOT be null.
  - Rating must be a value between 0.0 and 5.0 (Check Constraint).

Entity: Conversation

Description

This entity represents a unique identifier used to retrieve all stored Messages exchanged between a DogOwner and a DogSitter. While this entity does not store the messages itself, it serves as a persistent record of a Conversation that has been established between any two users at some point in time.

Attributes

Attribute	Type	Description	Constraints
id	Long	Used as a Primary Key	Primary Key
owner_id	Long	The unique id of the DogOwner	Foreign Key <Not Null>
sitter_id	Long	The unique id of the associated DogSitter	Foreign Key <Not Null>
created_at	Date	The date when the Conversation was created	Not Null

Relationships

- A **DogOwner** can have 0-to-Many **Conversations** with **DogSitters** (0:M)
- A **DogSitter** can have 0-to-Many **Conversations** with **DogOwners** (0:M)
- A **Conversation** is linked to and can contain 0-to-Many **Messages** (0:M)

Constraints & Business Rules

- *owner_id* is a foreign key and must NOT be null.
- *sitter_id* is a foreign key and must NOT be null.

Notes

- Both the Conversation and Message entity are essential entities for the live-messaging feature of this system.
- The creation of a Conversation occurs the moment a new Message has been sent by a DogOwner to a DogSitter who has not contacted each other before.

Entity: Message

Description

This entity stores all messages exchanged between users. The Conversation entity is used to associate and retrieve messages between a specific DogOwner and DogSitter.

The system's live messaging feature relies on this entity, functioning as a direct messaging (DM) system that enables real-time, and direct one-to-one communication. This helps reduce ambiguity and allows DogOwners and DogSitters to clarify details when arranging bookings for pet services.

Attributes

Attribute	Type	Description	Constraints
id	Long	Used as a Primary Key	Primary Key
conversation_id	Long	The unique id of a Conversation	Foreign Key <Not Null>
sender_id	Long	The unique id of the sender (user)	Foreign Key <Not Null>
content	String	Used to store the content of the message	Not Null, Max (200)
content_img_url	String	Used to load user-uploaded images	Nullable
is_seen	Bool	Determines whether the message was seen	Default (False)
time_sent	Date	Stores the time the message was sent	Not Null

Relationships

- Many **Messages** can belong to one **Conversation** (M:1)

Constraints & Business Rules

- *conversation_id* is a foreign key (from Conversation) and must NOT be null.
- *sender_id* is a foreign key (from User) and must NOT be null.
- *Content* field must not exceed 200 characters.
- The system must be able to determine whether the message has been seen.
- The system must be able to store the timestamp of the message sent.

Notes

- The *sender_id* is a foreign key from the primary keys from either the DogOwner (User) or the DogSitter classes.

USER INTERFACE DESIGN (UI/UX)

This section will define user stories, present high-fidelity wireframes, and outline the user flow interactions associated with the tasks described in those user stories. These designs will determine how users will interact with the system and how the system will conceptually look like up-front.

USER STORIES

As discussed earlier, the system has two primary user groups: **Dog Owners** and **Dog Sitters**. Dog Owners are focused on finding a suitable Dog Sitter and booking for pet services at times that are convenient for them.

Meanwhile, Dog Sitters are focused on efficiently managing multiple bookings from various Dog Owners through a simplified and organized interface. They need to be able to accept or reject booking requests, handle concurrent bookings, and view detailed pet profiles submitted by requesters.

Dog Owner

- As a Dog Owner, I want to search for available Dog Sitters by date & time, so that I can find someone who matches my schedule.
- As a Dog Owner, I want to be able to find dog sitters within my area.
- As a Dog Owner, I want to be able to register my pets on the app for servicing.
- As a Dog Owner, I want to be able view a Dog Sitter's profile and reviews, so that I can assess their credibility before booking.
- As a Dog Owner with multiple dogs, I want to be able to select a specific dog when submitting a booking request, so that the Dog Sitter knows which one of my pets require service.
- As a Dog Owner, I want to modify or cancel a booking so that I can adjust my plans if needed.
- As a Dog Owner, I want to be able to directly message with Dog Sitters to clarify service details.
- As a Dog Owner, I want to be able to leave a review for a Dog Sitter, so that I can share my experience with others.

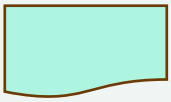
Dog Sitter

- As a Dog Sitter, I want to easily view and manage multiple bookings and requests through a clear, simplified interface.
- As a Dog Sitter, I want to be able to easily set my sitting preferences, so that Dog Owners can respect what I am willing to do and what I will not.
- As a Dog Sitter, I want to be able to easily register and promote my services, so Dog Owner can see what I can provide for their pets.
- As a Dog Sitter, I want to be able to set my availability, so that Dog Owners can book me for open time slots.
- As a Dog Sitter, I want to be able to receive booking requests, so that I can review and decide whether to accept them or not.
- As a Dog Sitter, I want to be able to accept or decline booking requests, so that I can manage my schedule.
- As a Dog Sitter, I want to be able to view details about the requested service and the associated pets, so that I can make an informed decision.
- As a Dog Sitter, I want to be able to message Dog Owners, so that I can discuss requirements or address any confusions before accepting a booking.

USER FLOW DIAGRAMS

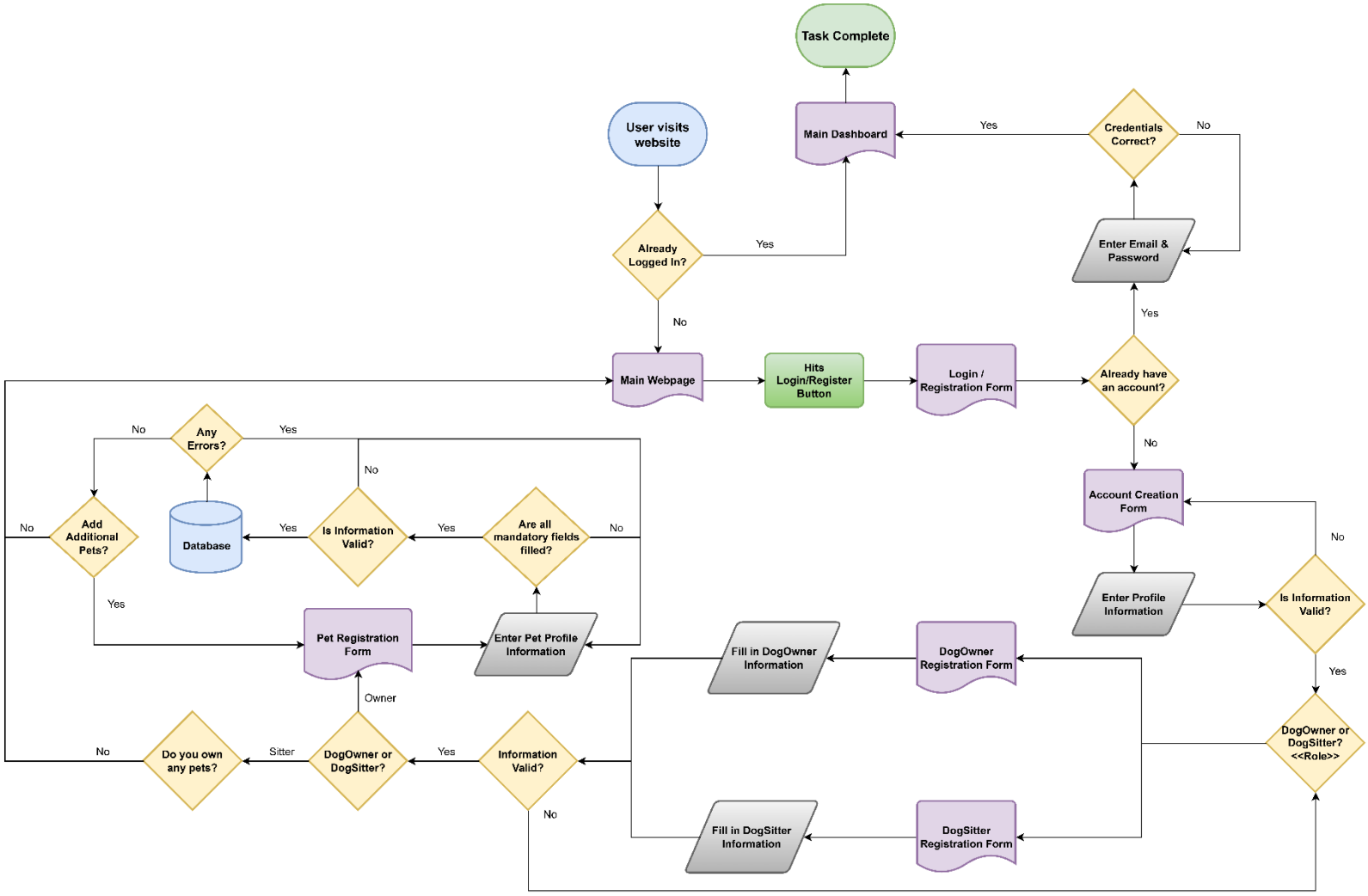
User flow diagrams visually represent the sequence of steps a user must take in order to complete a specific task within a system. They illustrate how users navigate through different screens, make decisions, and interact with the system, helping to clarify the overall user journey and identify potential usability issues. This section will go over user flows for the scenarios described in the user stories.

Symbols and Definitions

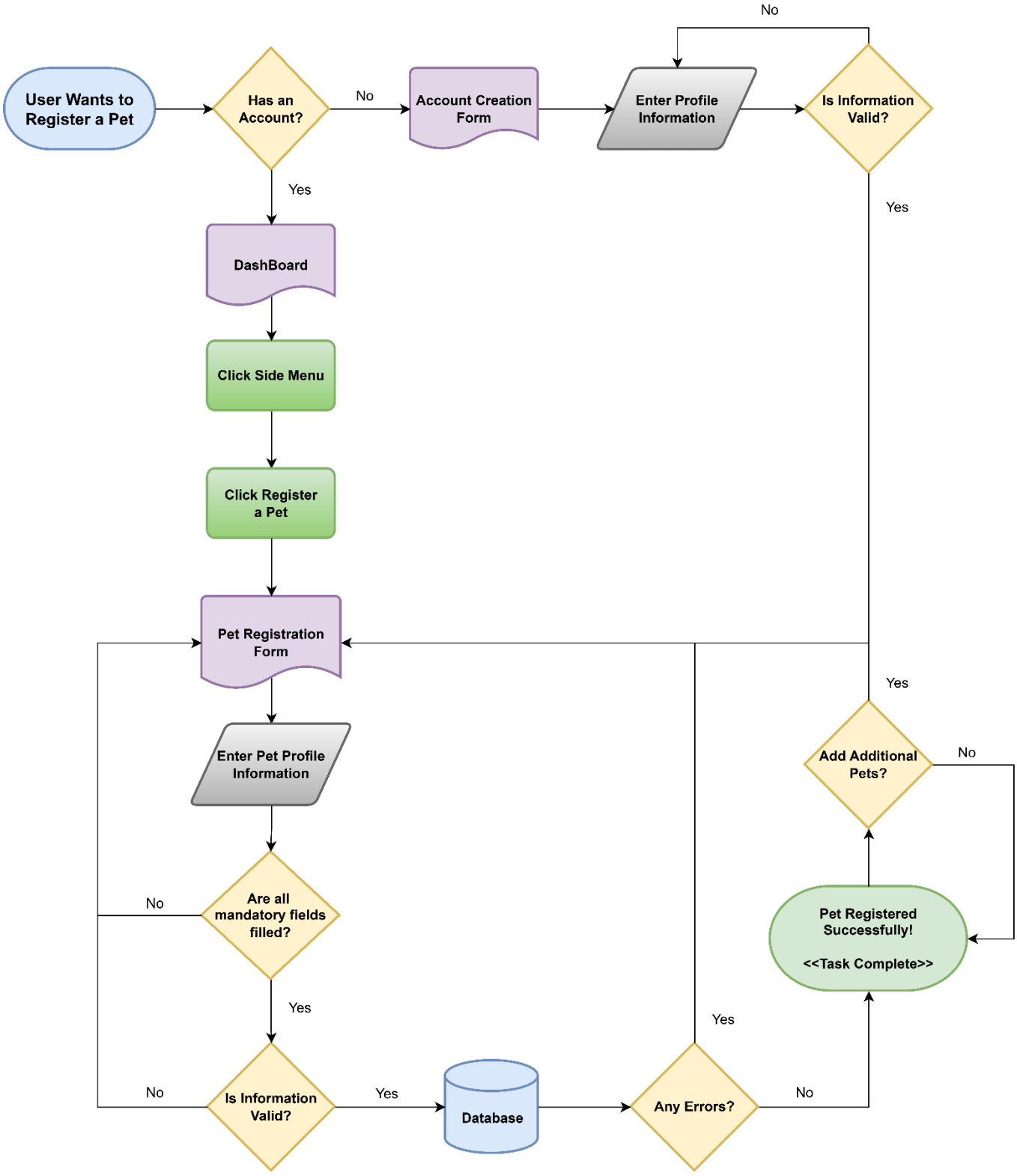
Symbol	Name	Function
	Pill-Shape <Start/End>	Represents the start or end point of the user flow
	Rectangle <Action/Process>	Represents a user performing an action or process
	Diamond <Decision/Condition>	Represents a condition or a decision point
	Parallelogram <Input/Output>	Represents data entering or leaving the system
	Upside-Down Wave <Document/Page>	Represents a printed document or page

User Flow (All Users)

Login / Registration

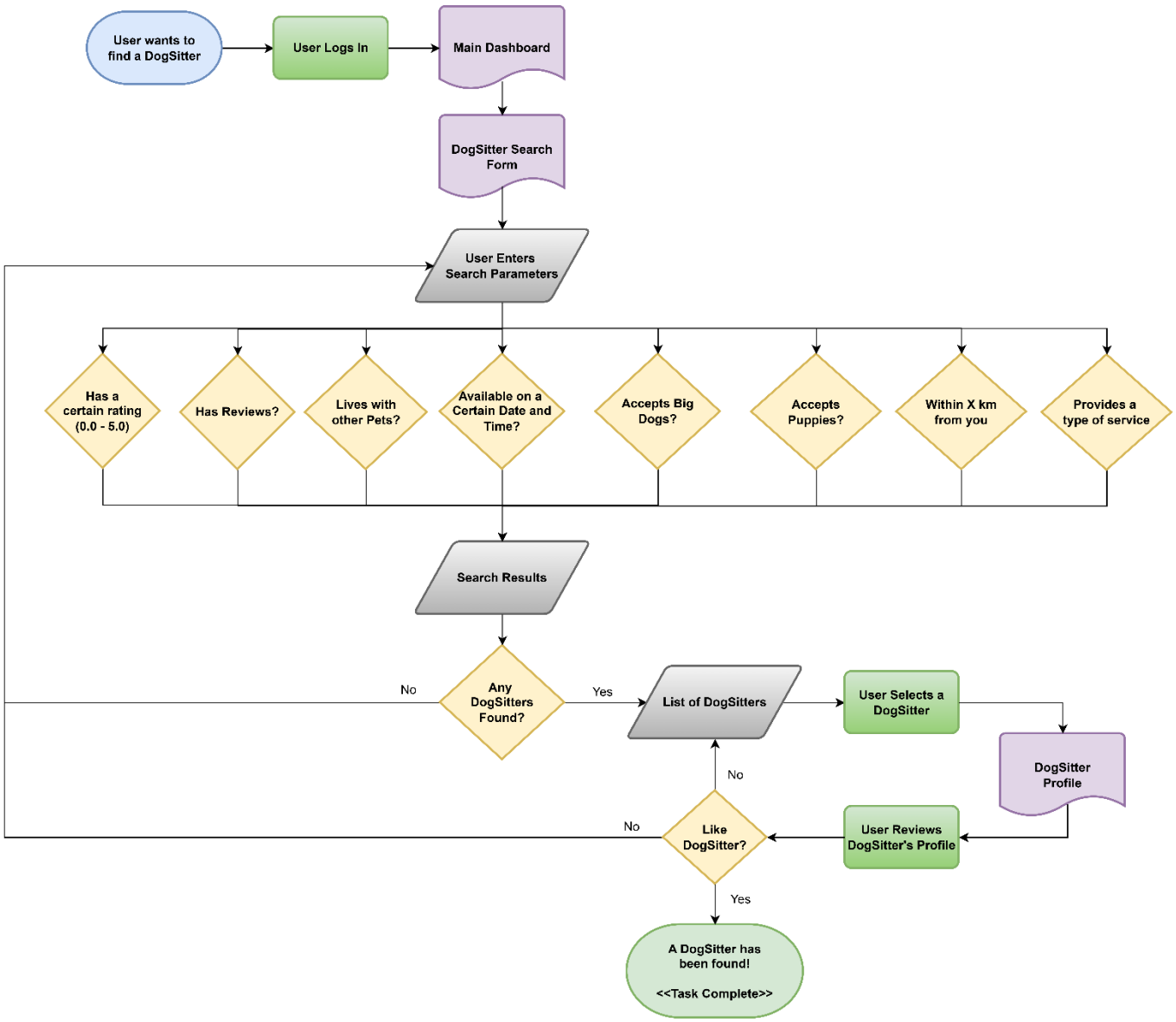


Registering a Pet

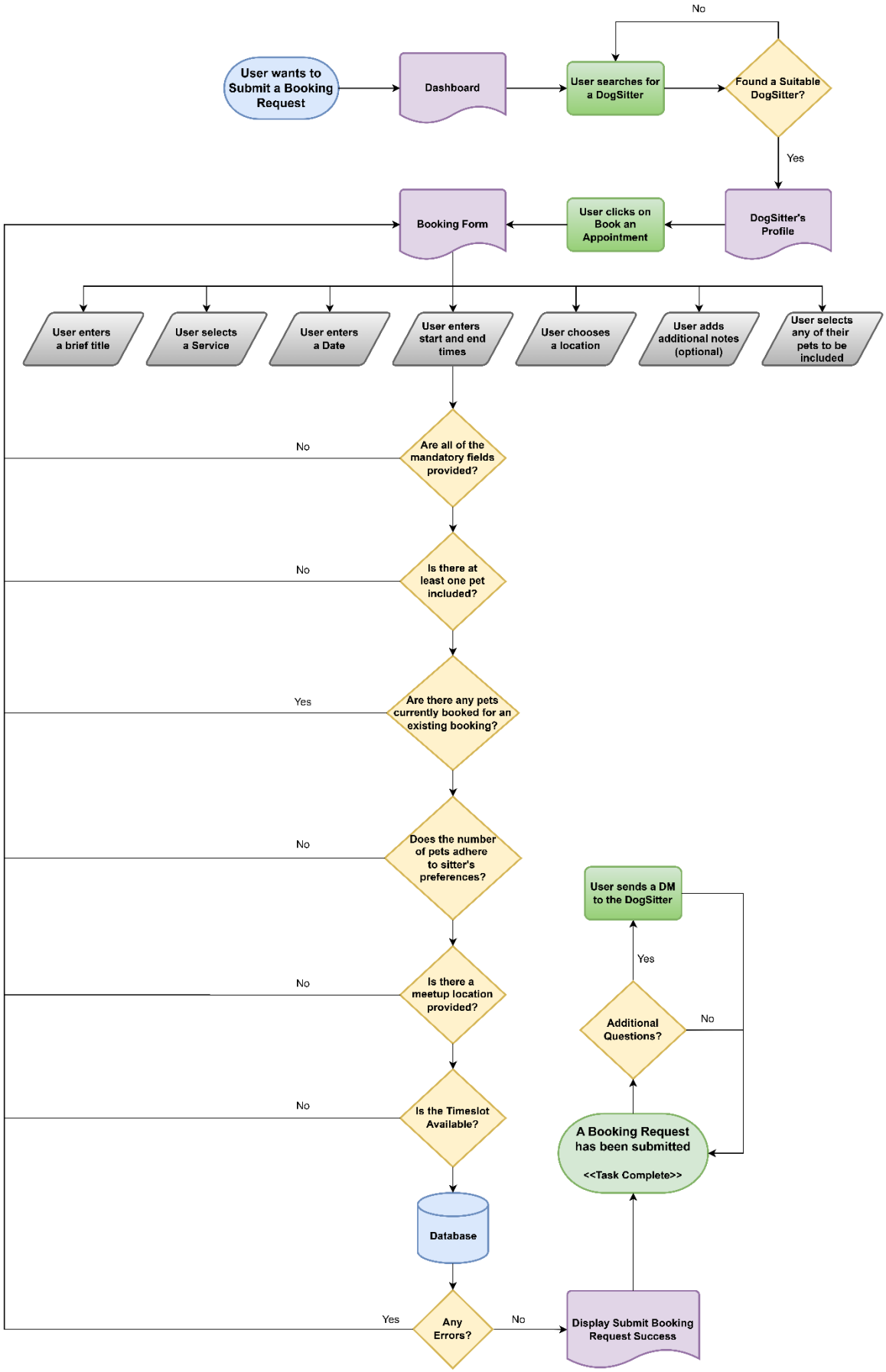


User Flow (Dog Owner)

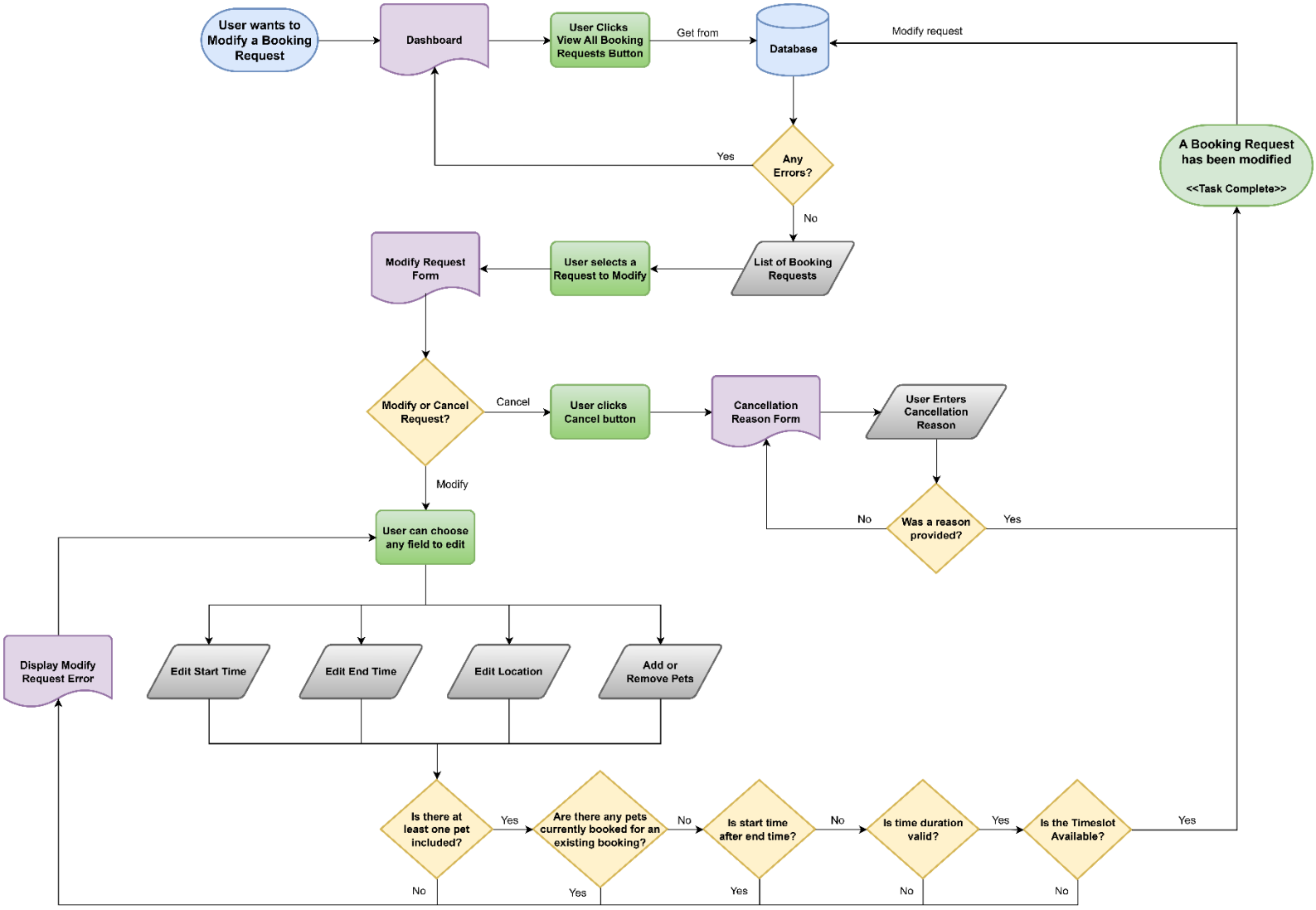
Finding a Dog Sitter



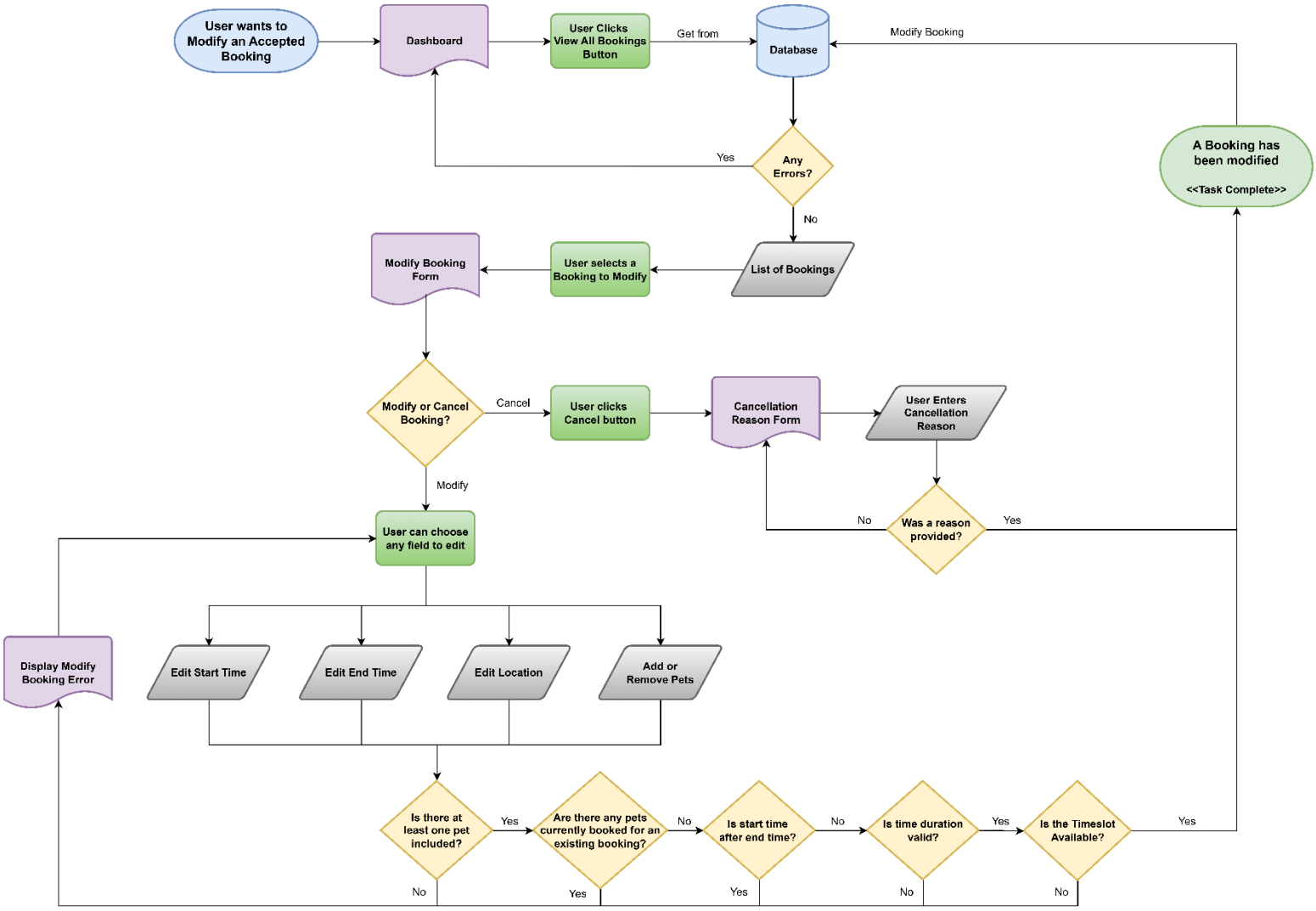
Submitting a Booking Request



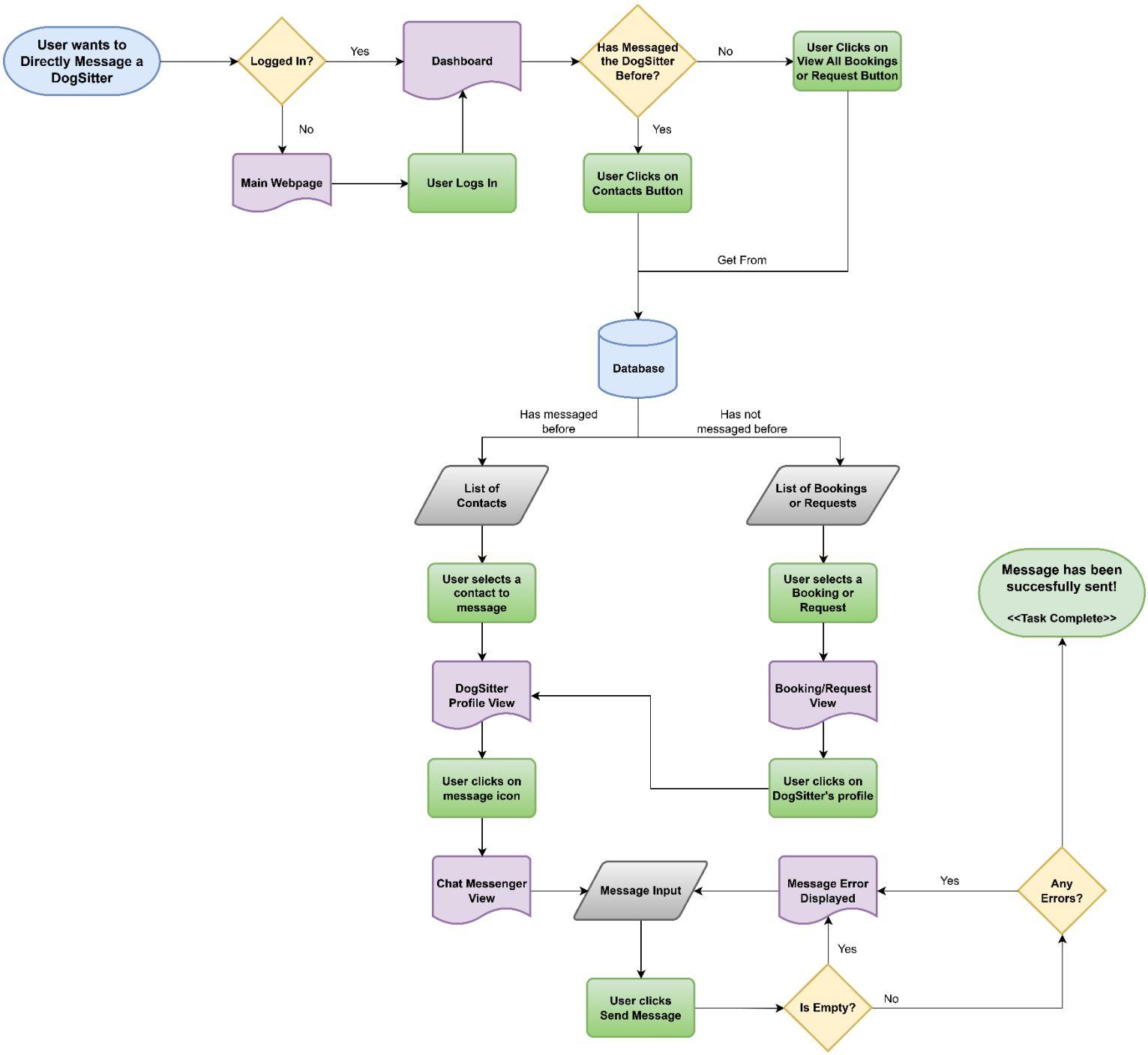
Modifying a Booking Request



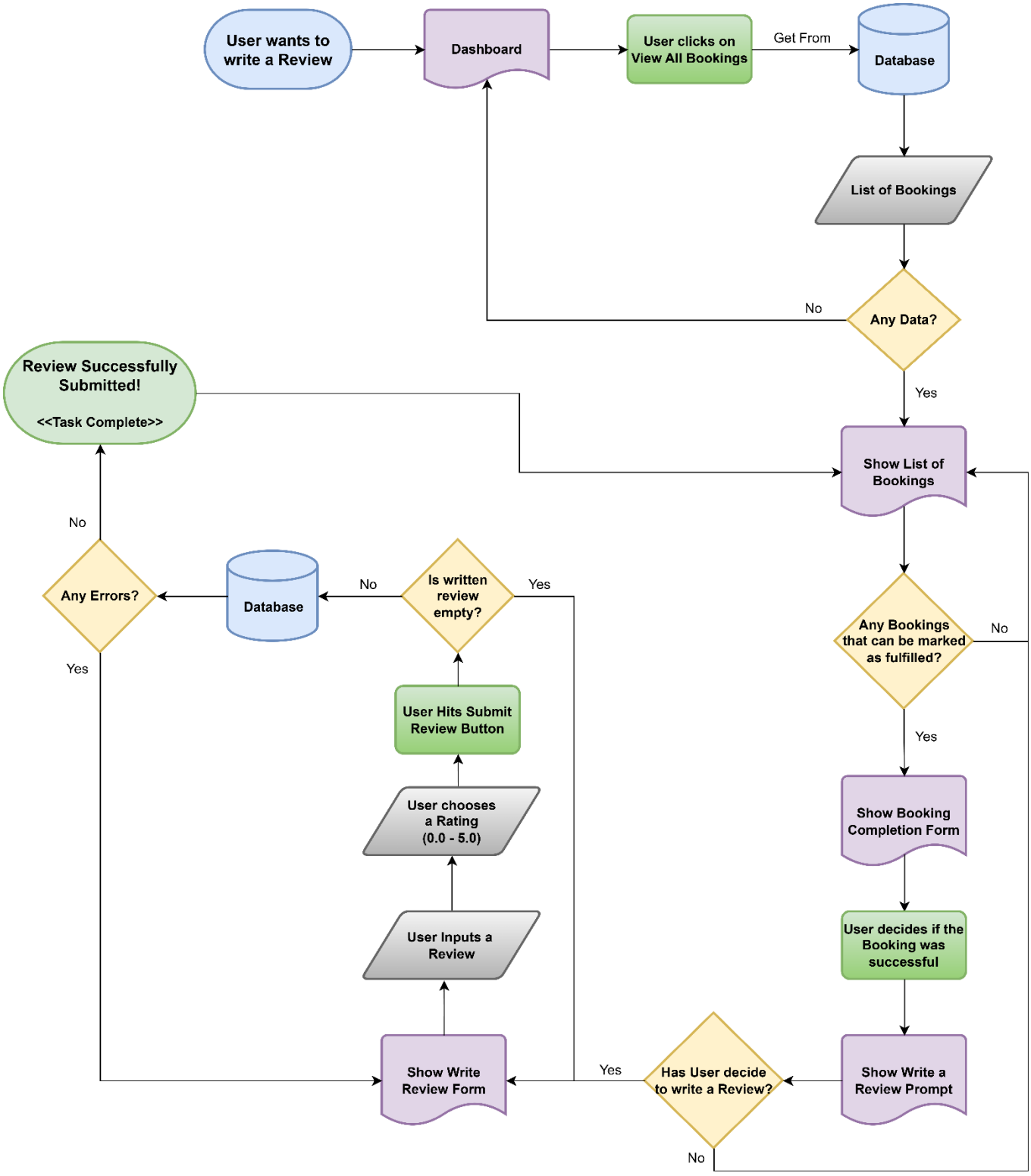
Modifying an Accepted Booking



Direct Messaging a Dog Sitter



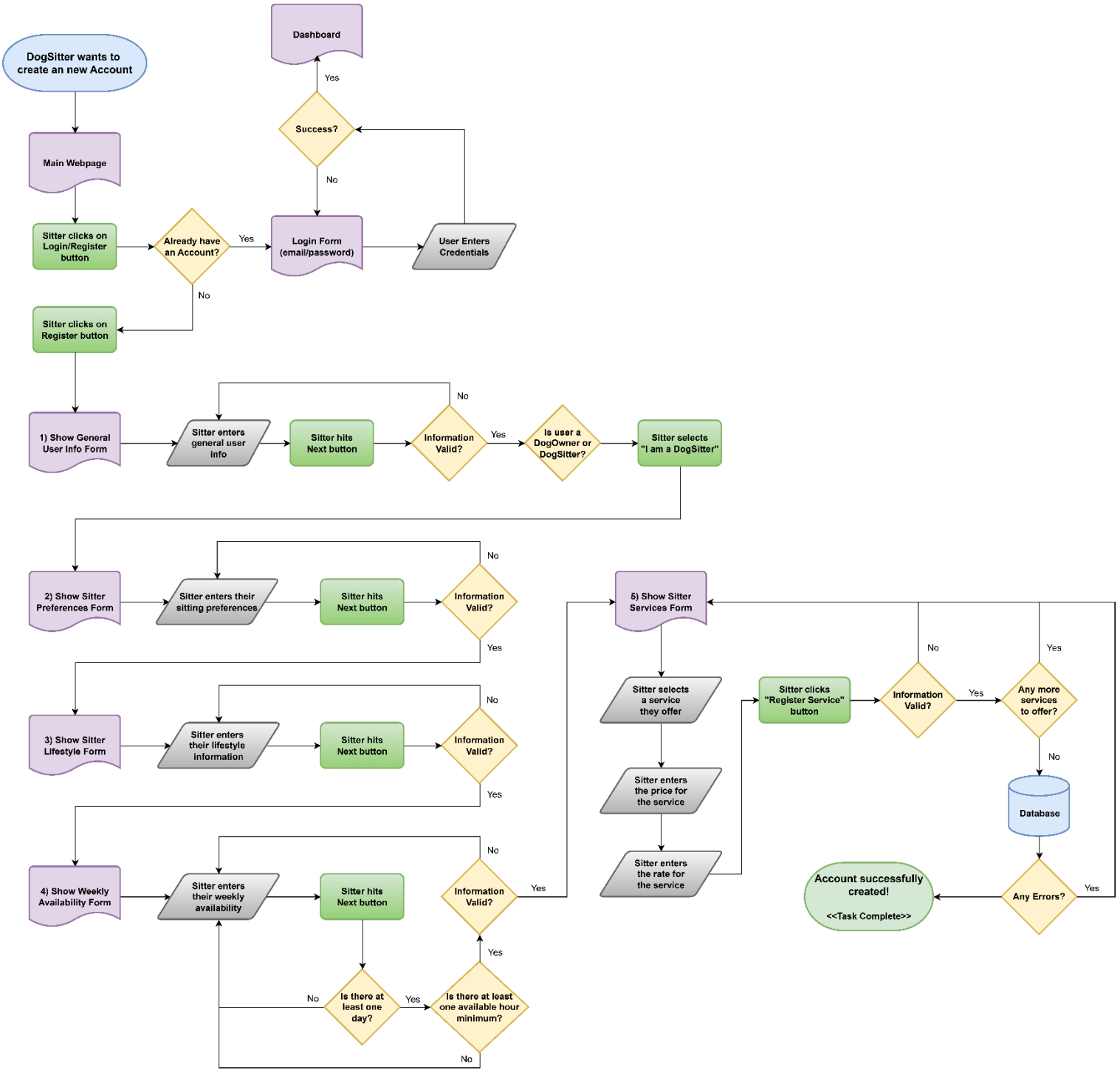
Writing a Review



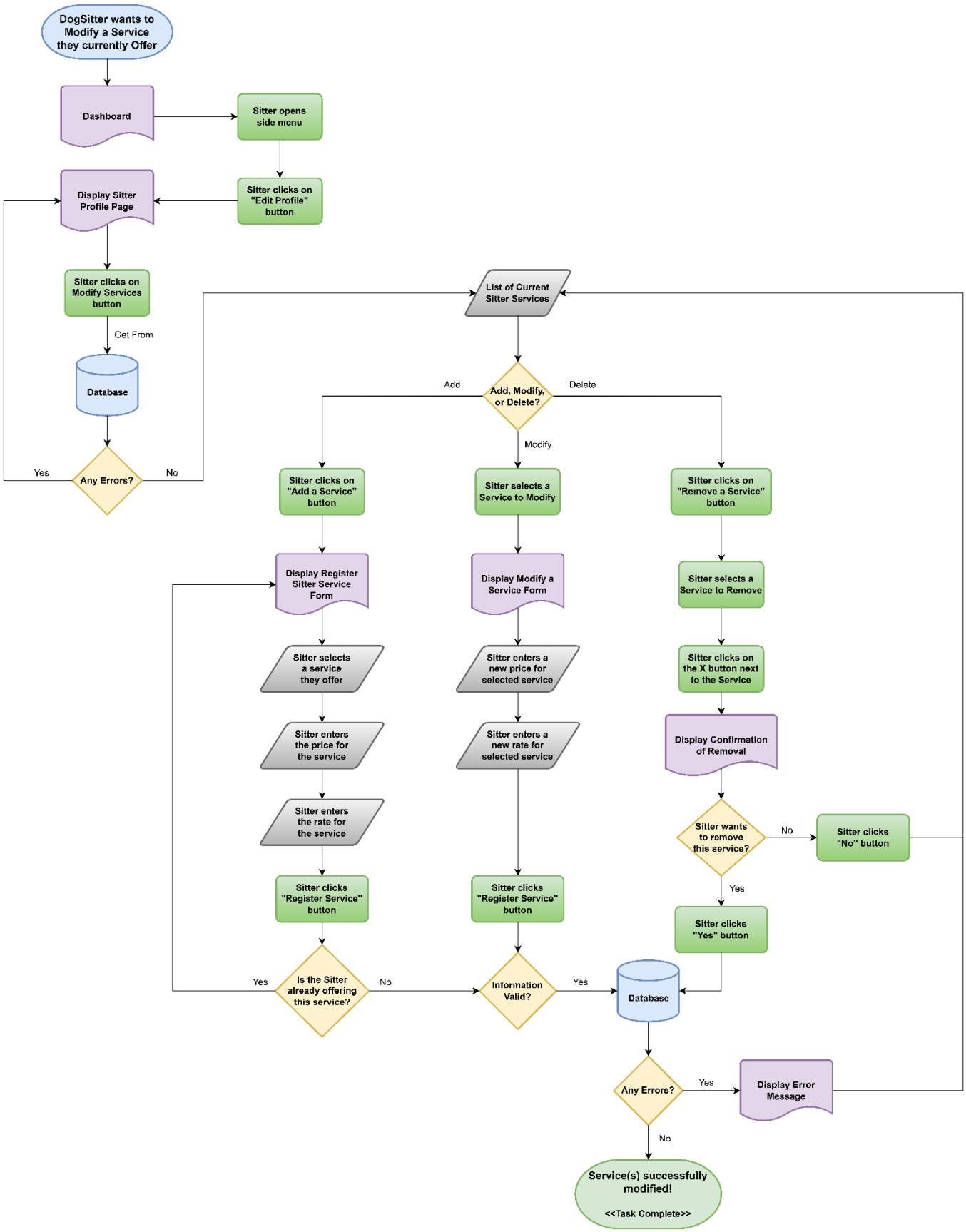
User Flow (Dog Sitter)

Profile Setup

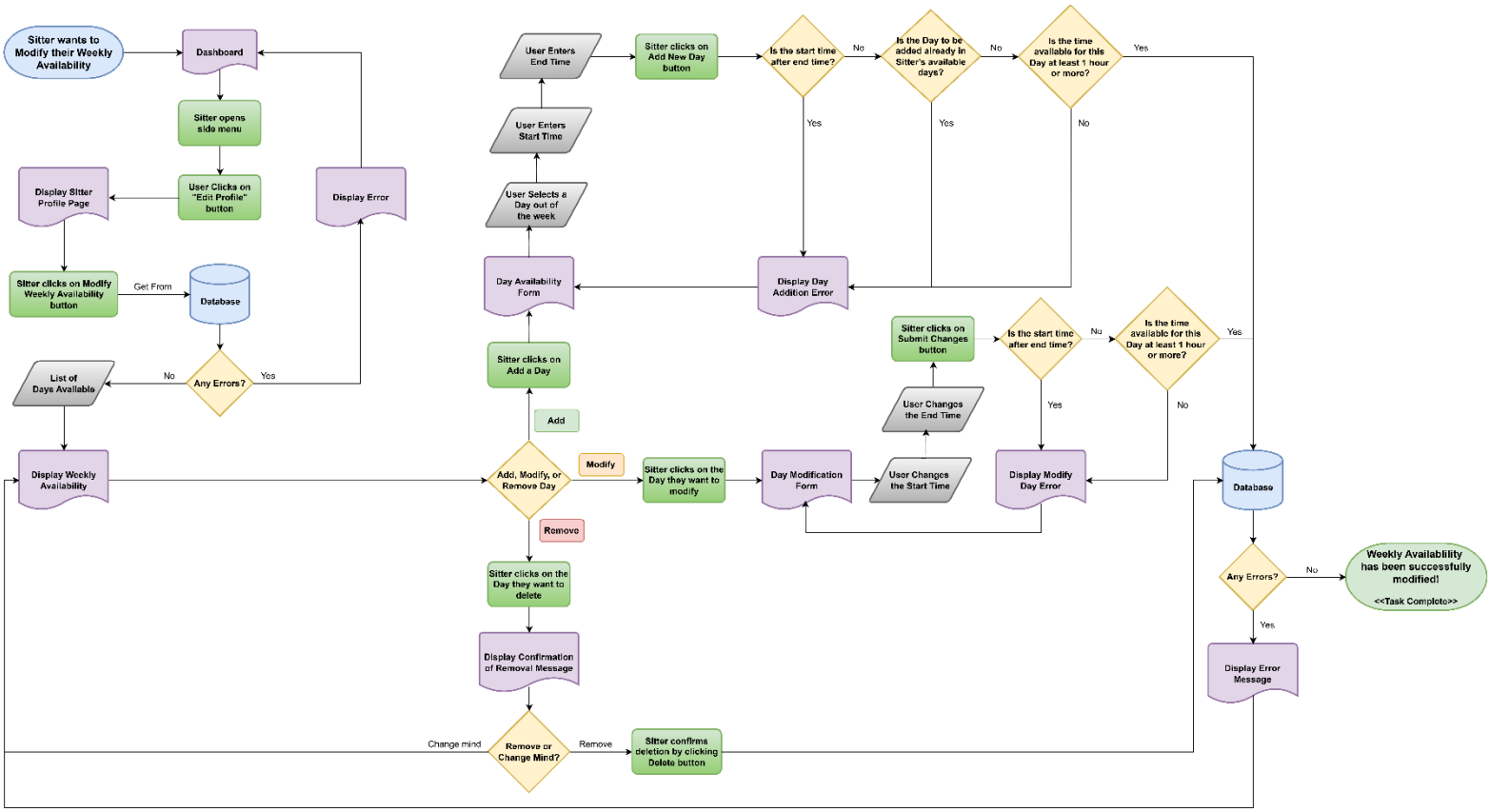
Note: This flow includes a new account setup with *SitterPreferences, SitterLifestyle, SitterServices* and Availability being set by the Dog Sitter.



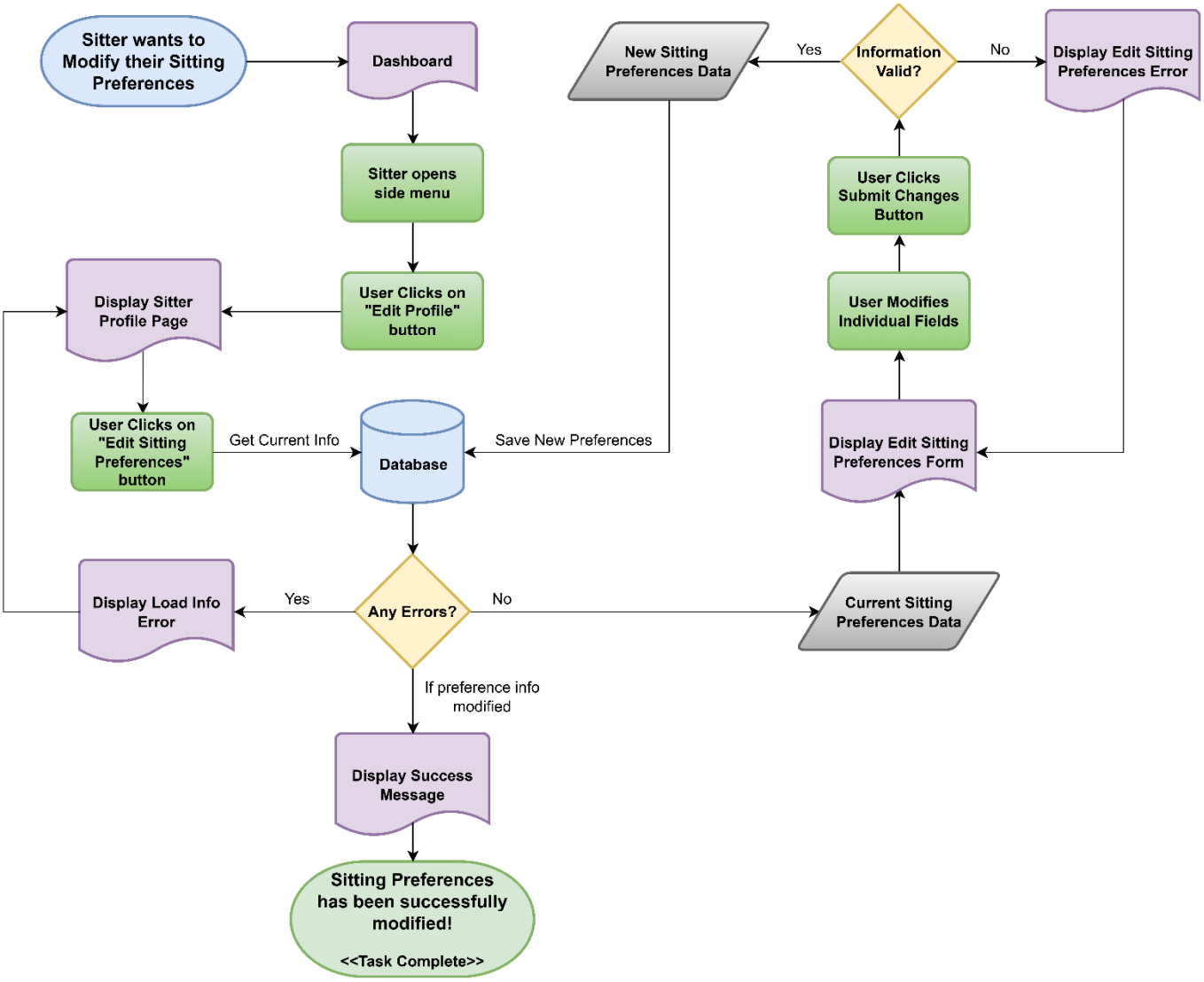
Modifying Services Offered



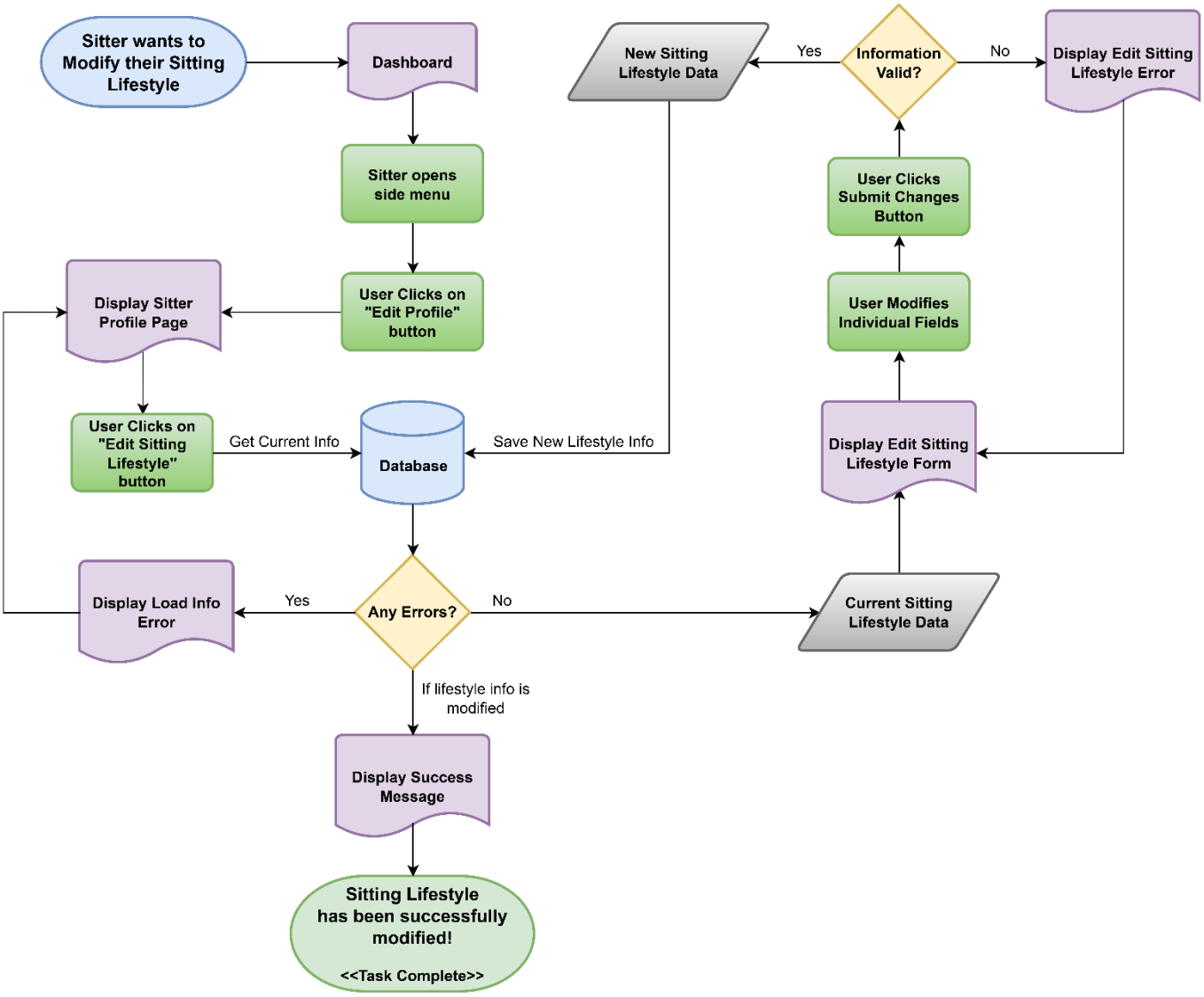
Modifying Weekly Availability



Modifying Sitter Preferences

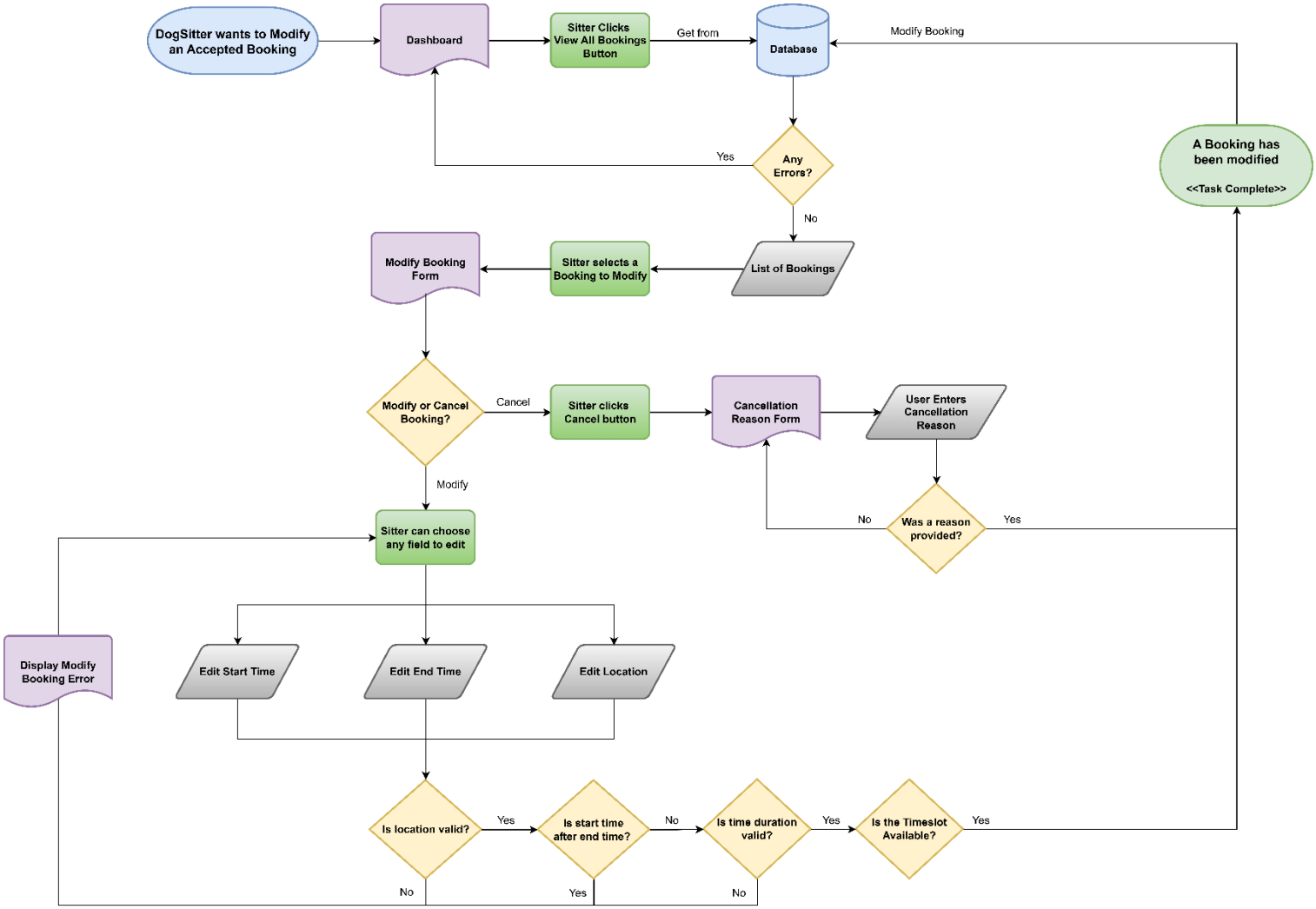


Modifying Sitter Lifestyle

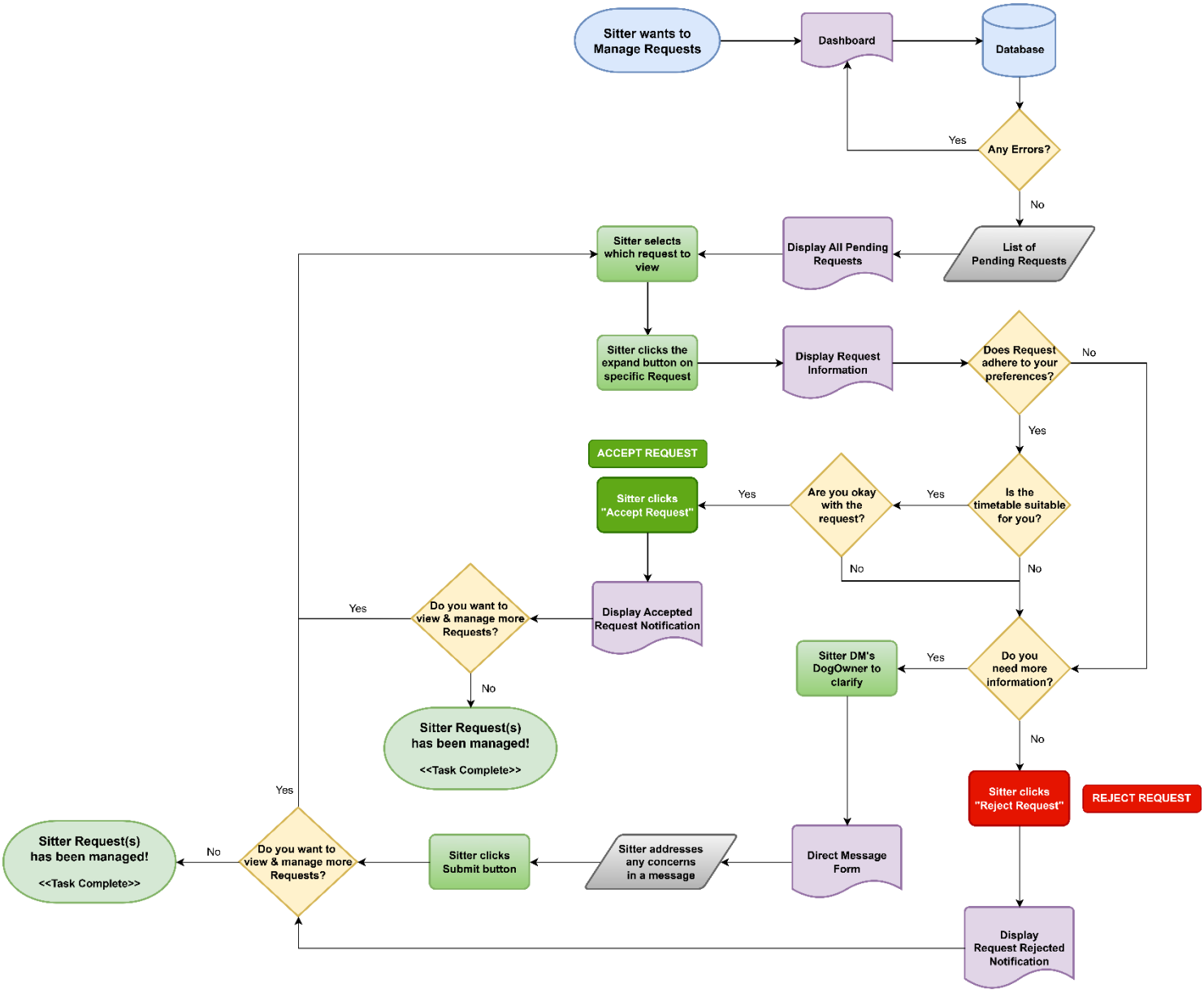


Modifying an Accepted Booking

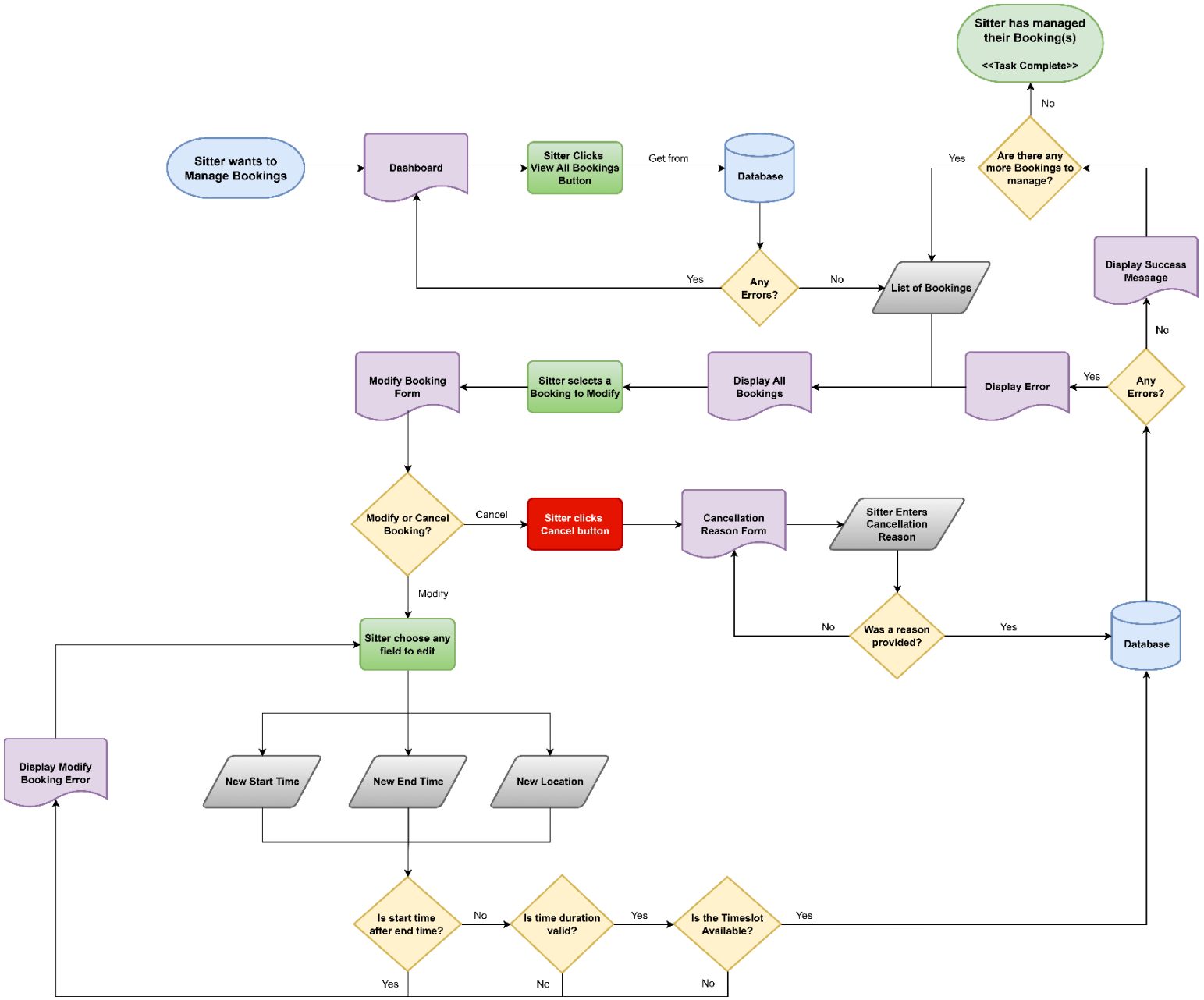
Note: DogSitters should not be able to modify a request since they did not initially make it. They can only refuse or accept them. Therefore, there is no “Modify Request” user flow.



Request Management



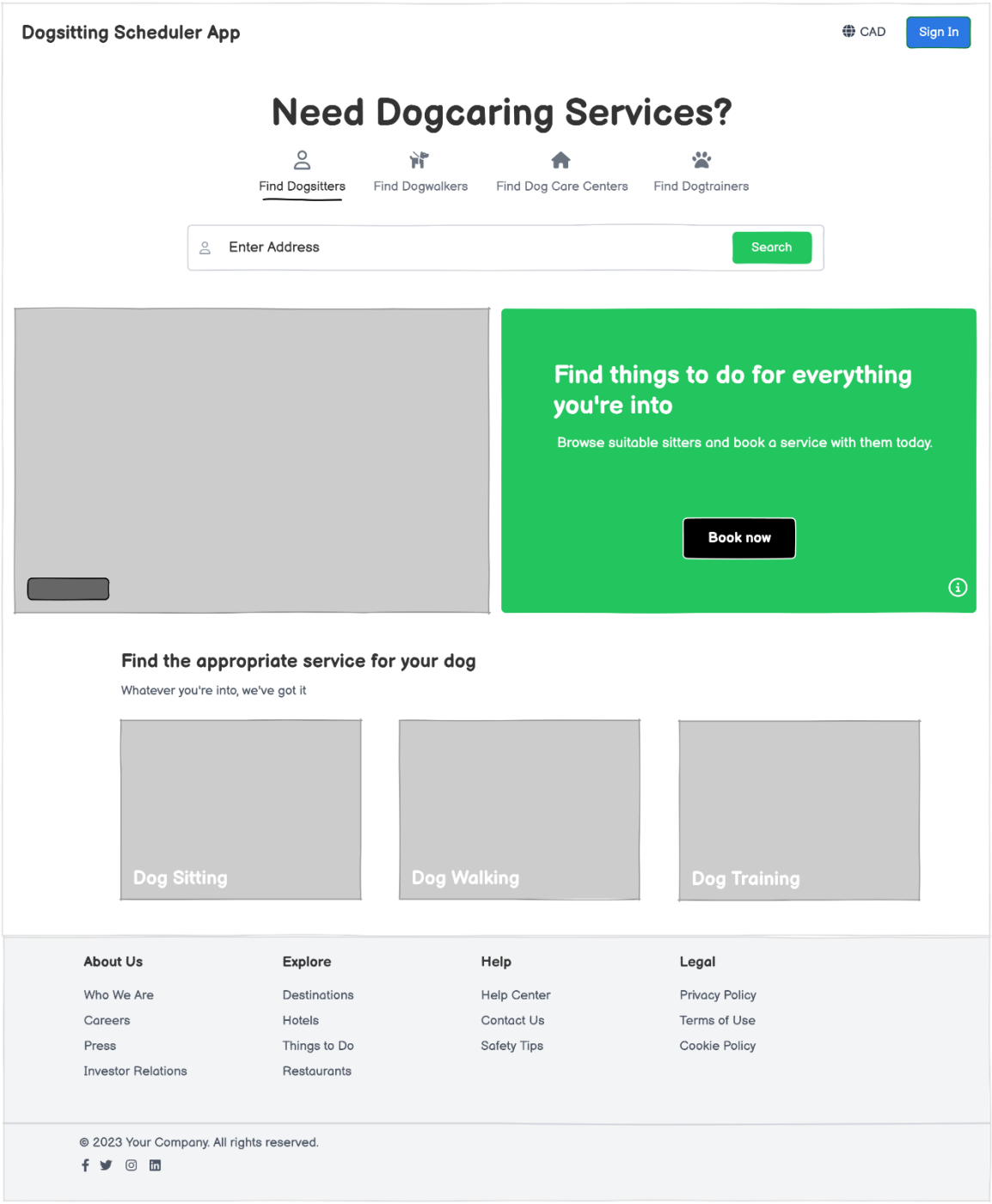
Booking Management



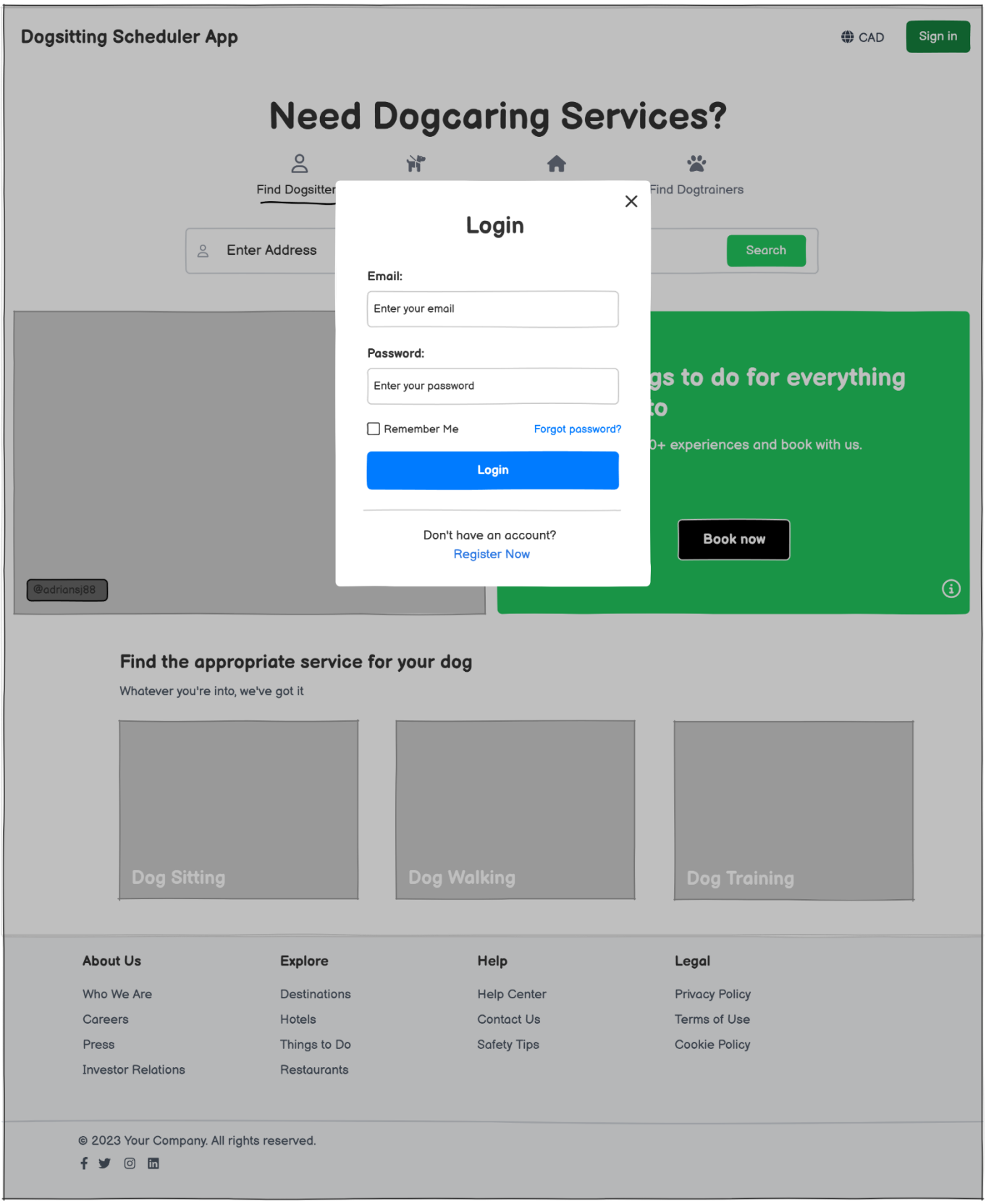
WIREFRAMES

This section will present some mid-fidelity wireframes that illustrate the initial designs and overall layout of the system. These wireframes are intended to provide a clearer understanding of the system's structure, navigation, flow, and core functionalities from the perspectives of both Dog Owners and Dog Sitters. Please note that the designs shown is not final and can change during development.

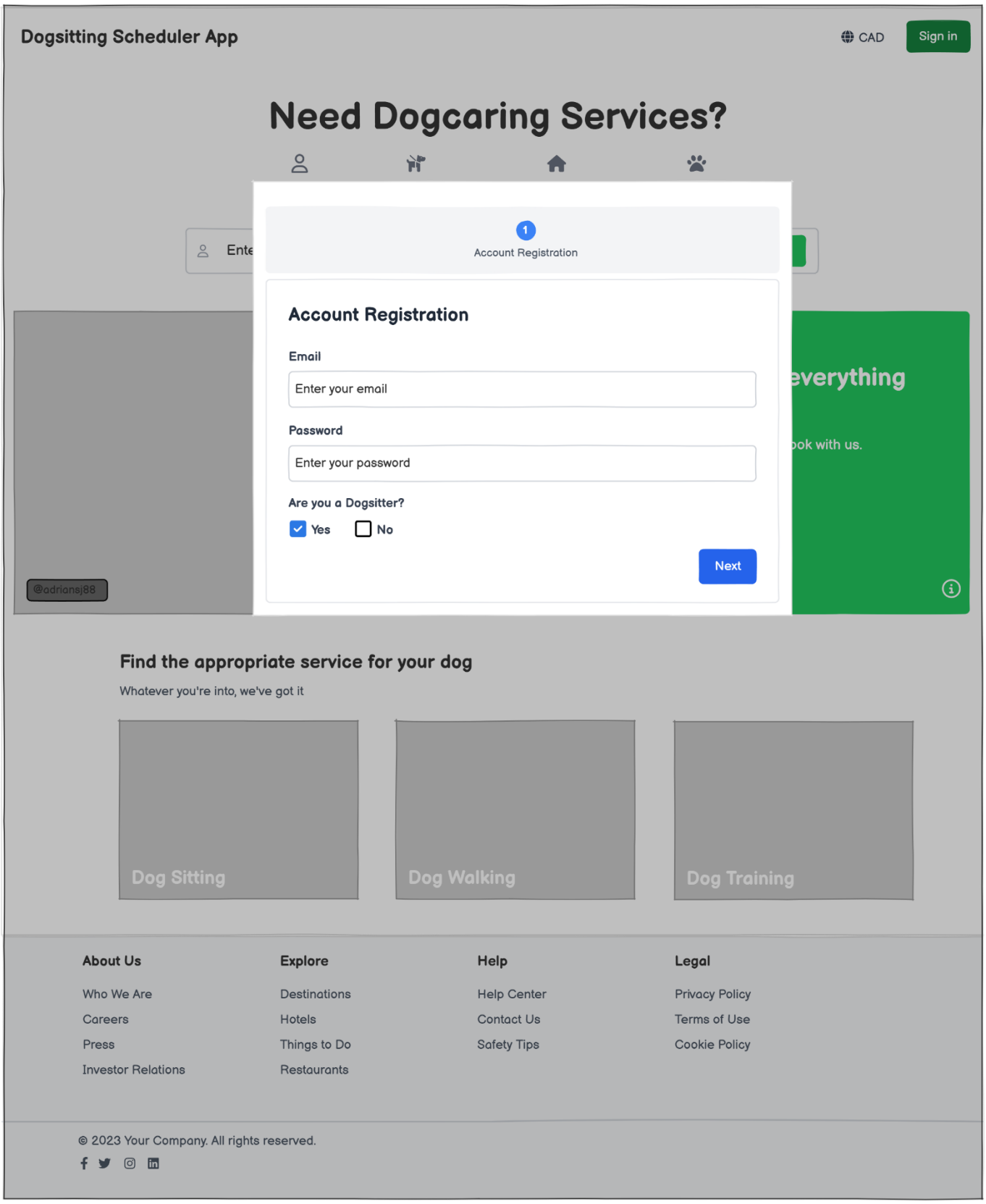
Splash Page



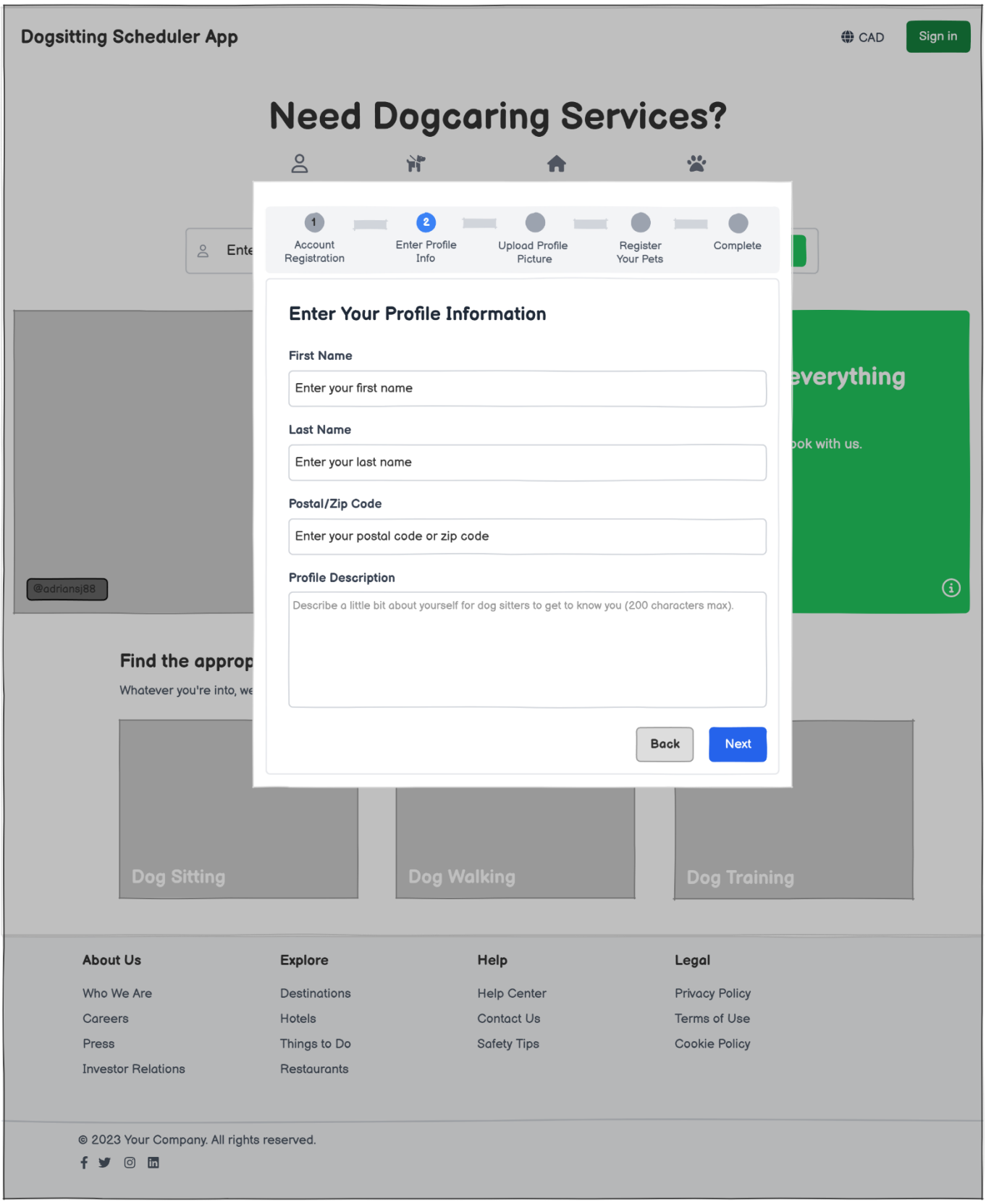
Login


Dogsitting Scheduler App
CAD
Sign in
Need Dogcaring Services?
Find Dogsitter
Find Dogtrainers
Enter Address
Search
Login
Email:
Enter your email
Password:
Enter your password
Remember Me
Forgot password?
Login
Don't have an account?
Register Now
gs to do for everything
o
0+ experiences and book with us.
Book now
@adriansj88
Find the appropriate service for your dog
Whatever you're into, we've got it
Dog Sitting
Dog Walking
Dog Training
About Us
Who We Are
Careers
Press
Investor Relations
Explore
Destinations
Hotels
Things to Do
Restaurants
Help
Help Center
Contact Us
Safety Tips
Legal
Privacy Policy
Terms of Use
Cookie Policy
© 2023 Your Company. All rights reserved.

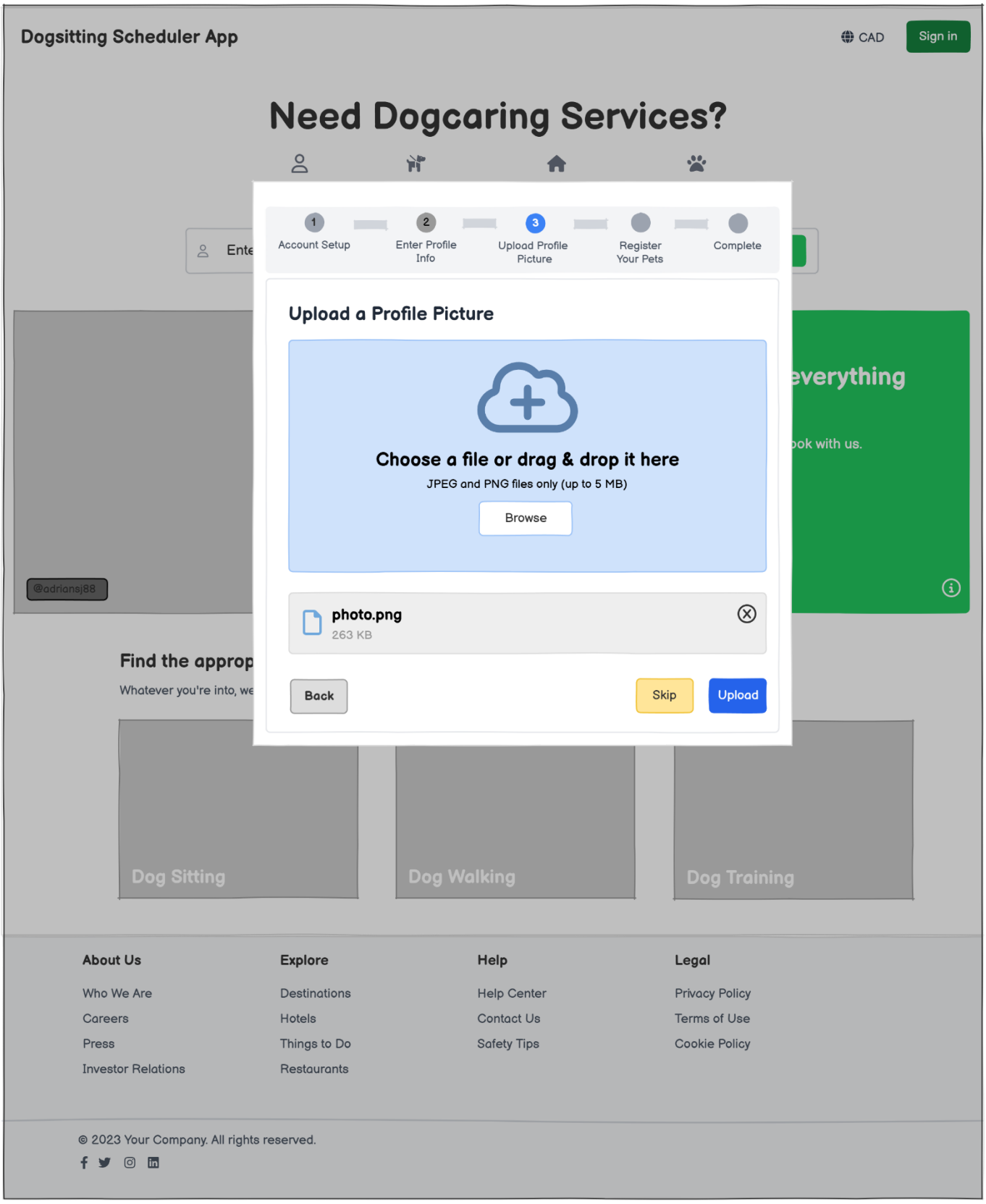
All Users: Account Registration (Email & Password)



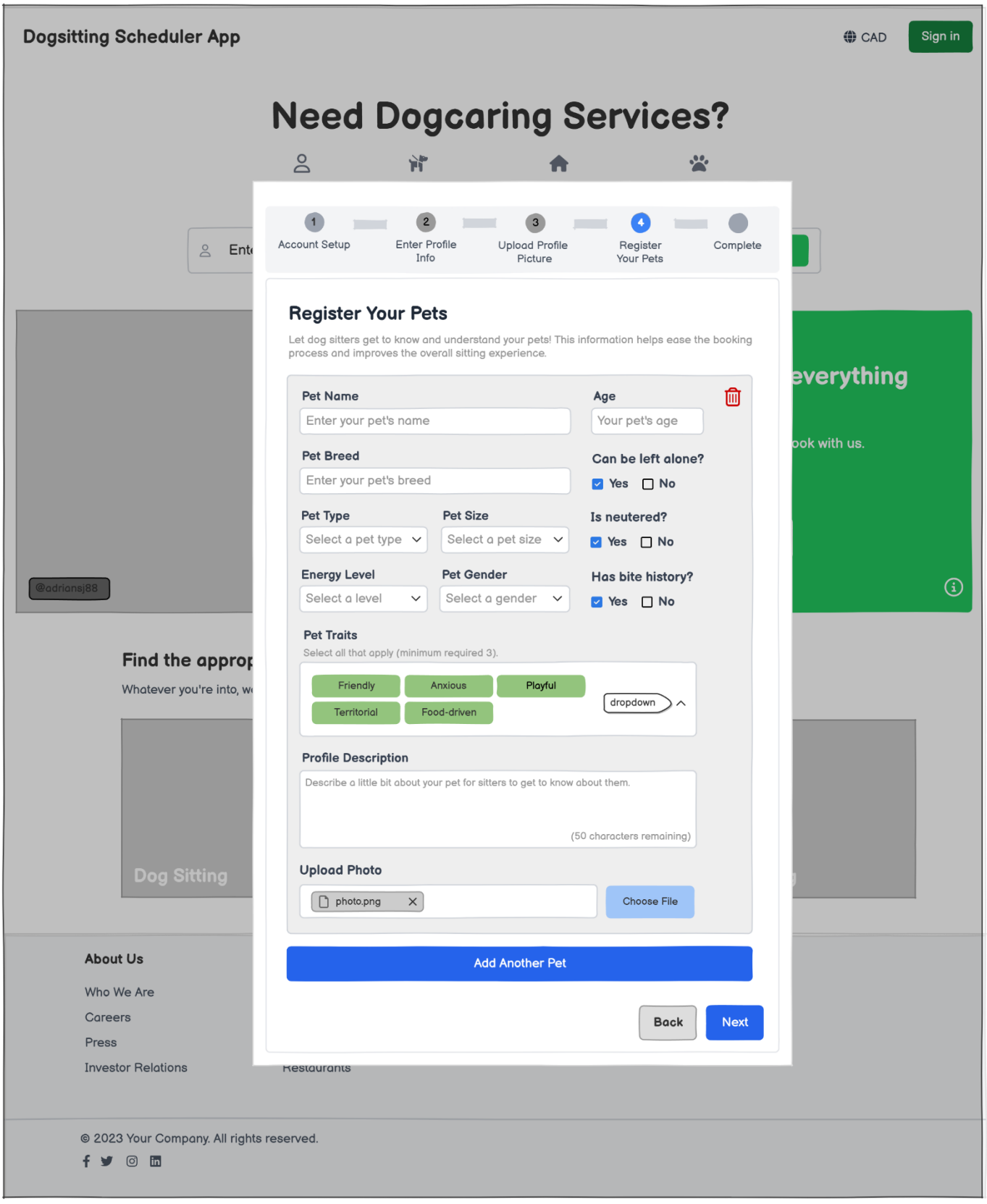
Dog Owner: Account Registration (Entering Profile Information)



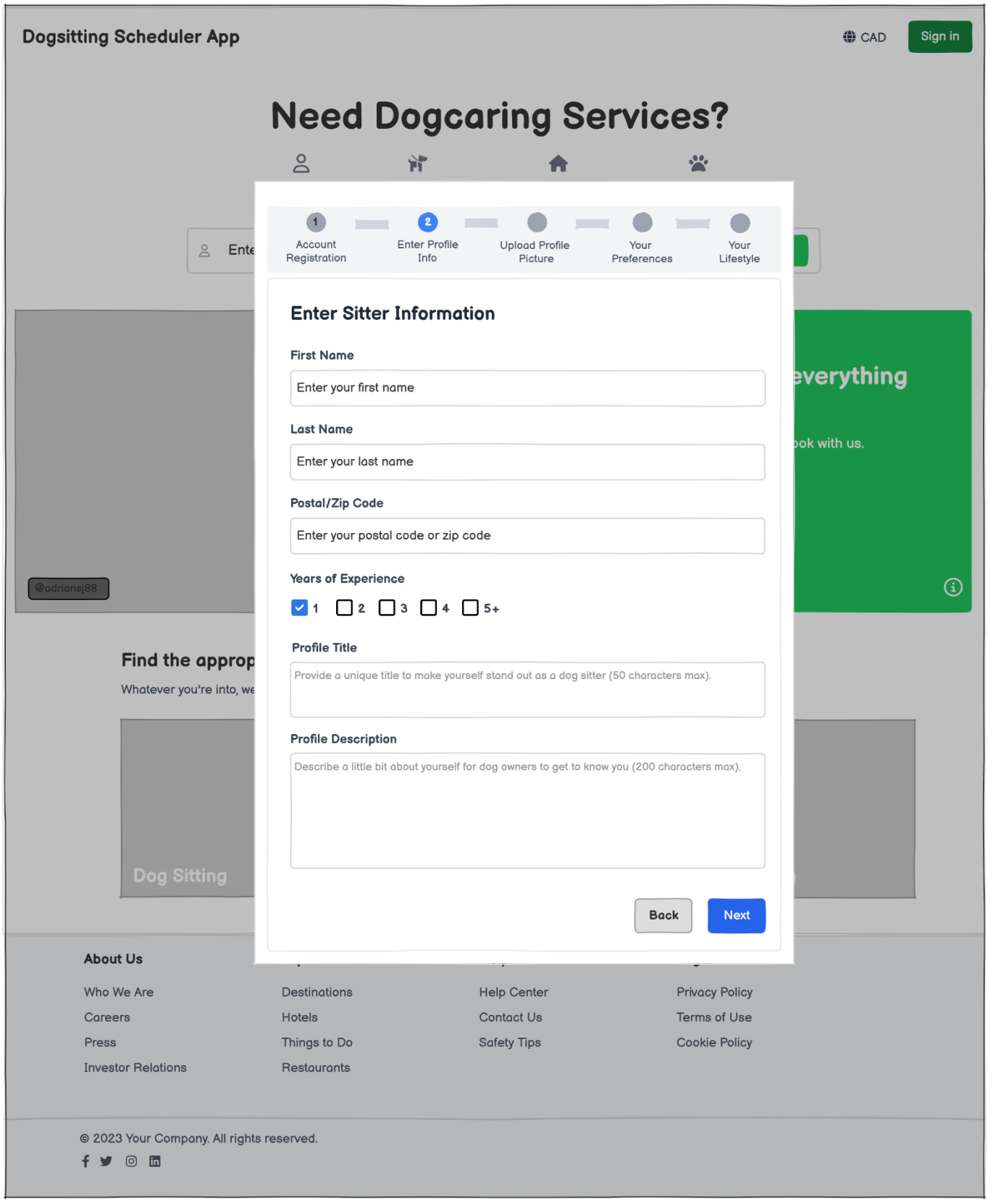
Dog Owner: Account Registration (Uploading Profile Picture)



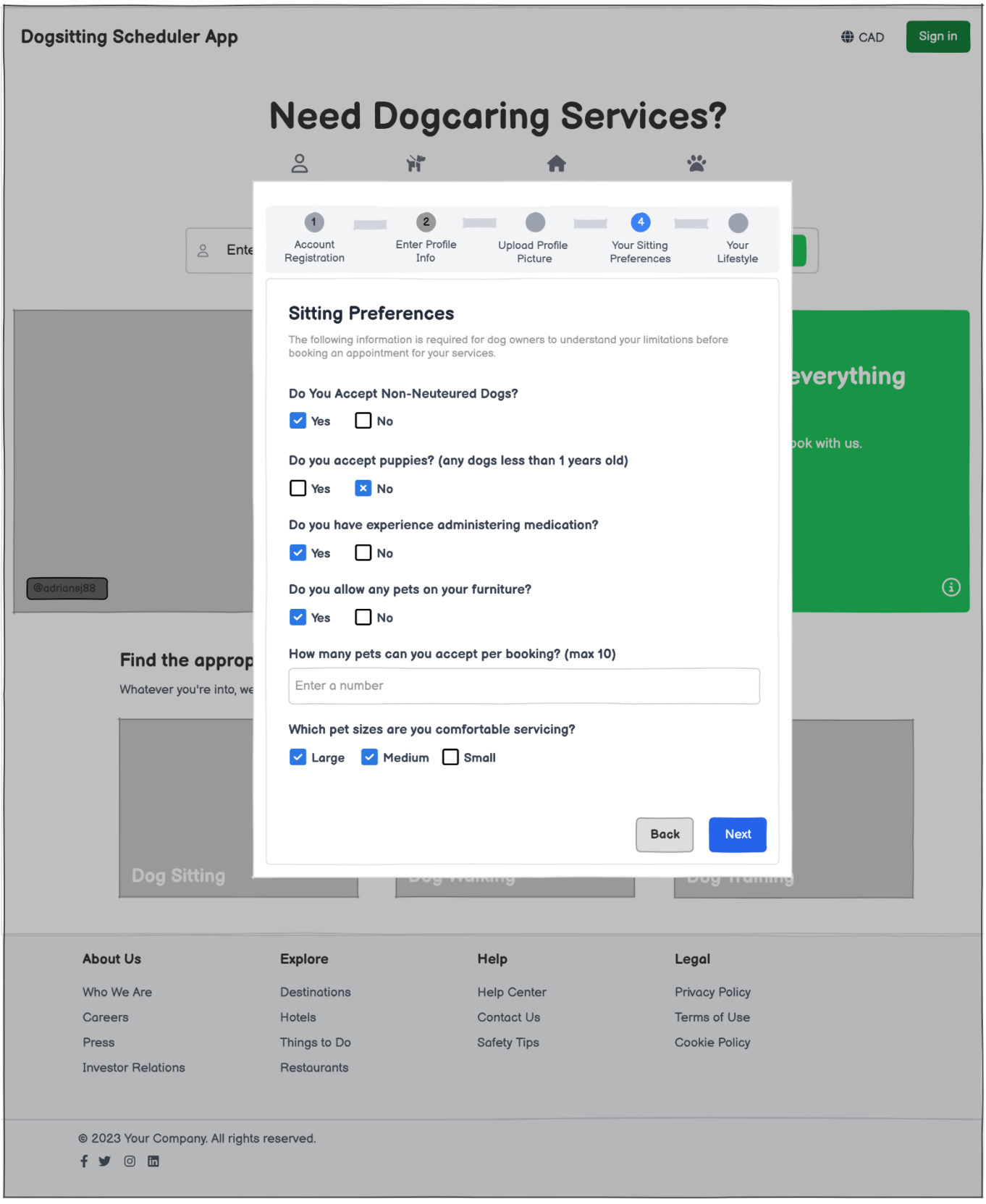
Dog Owner: Account Registration (Registering Pets)



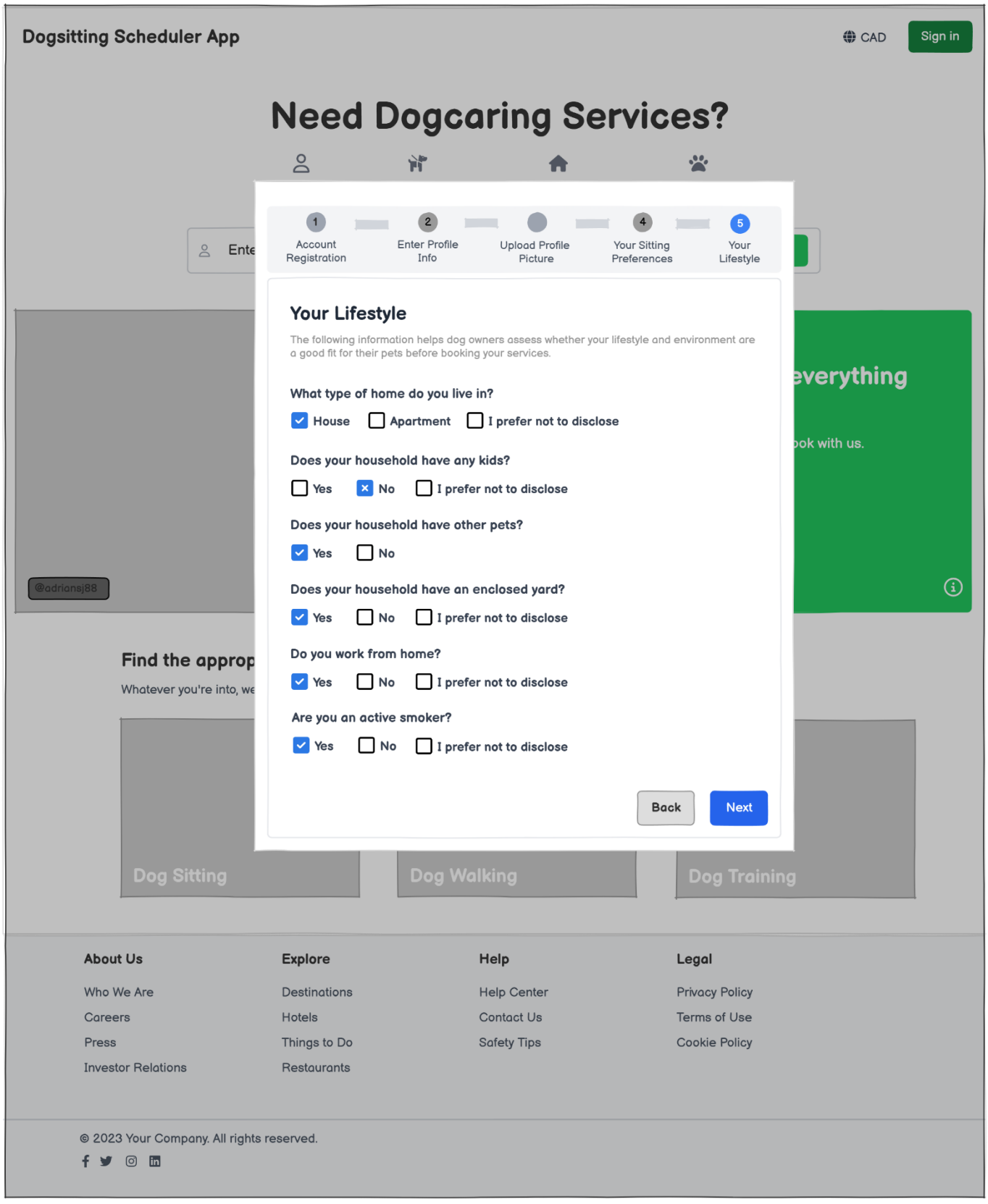
Dog Sitter: Account Registration (Entering Sitter Information)



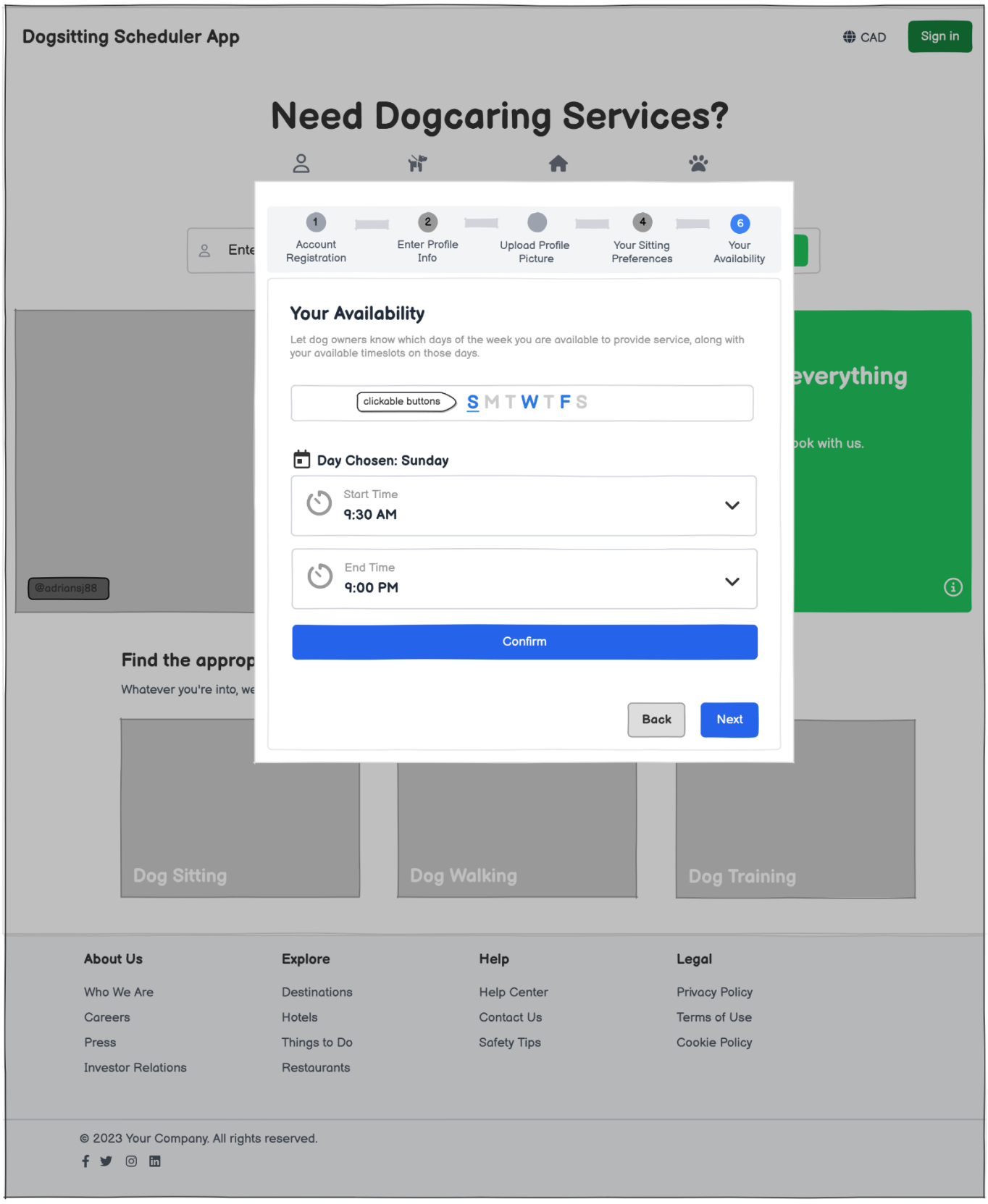
Dog Sitter: Account Registration (Entering Sitter Preferences)



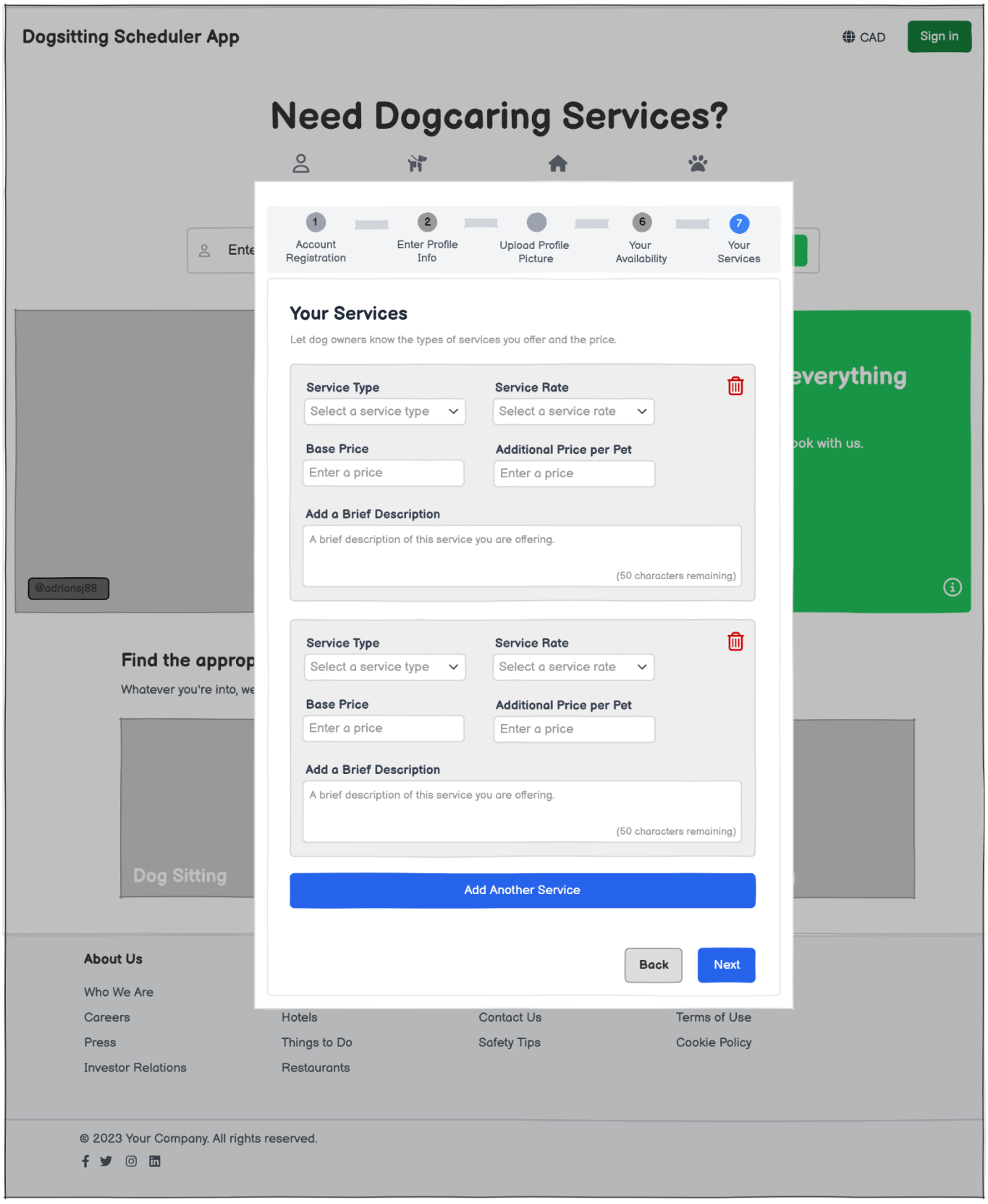
Dog Sitter: Account Registration (Entering Sitter Lifestyle)



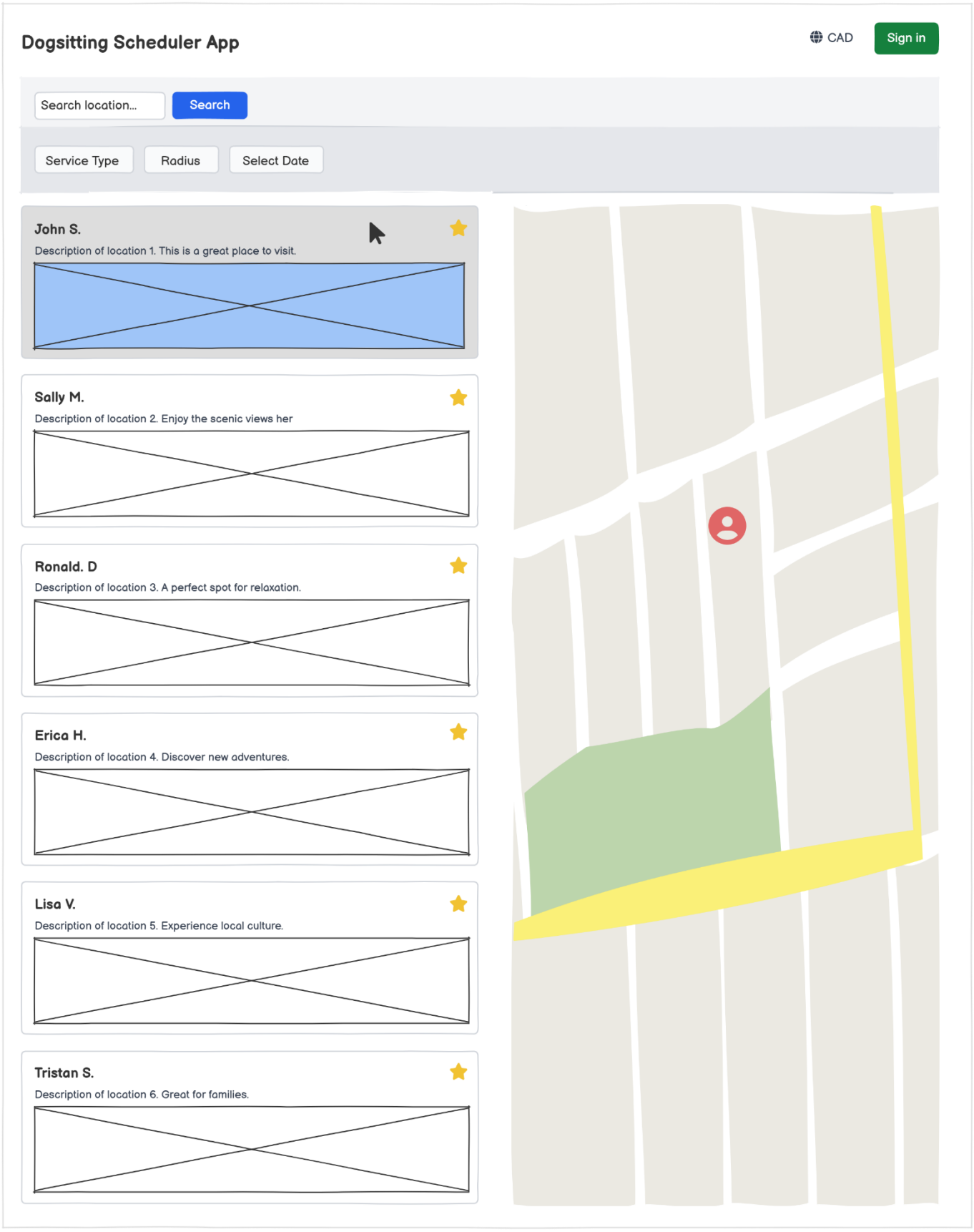
Dog Sitter: Account Registration (Entering Weekly Availability)



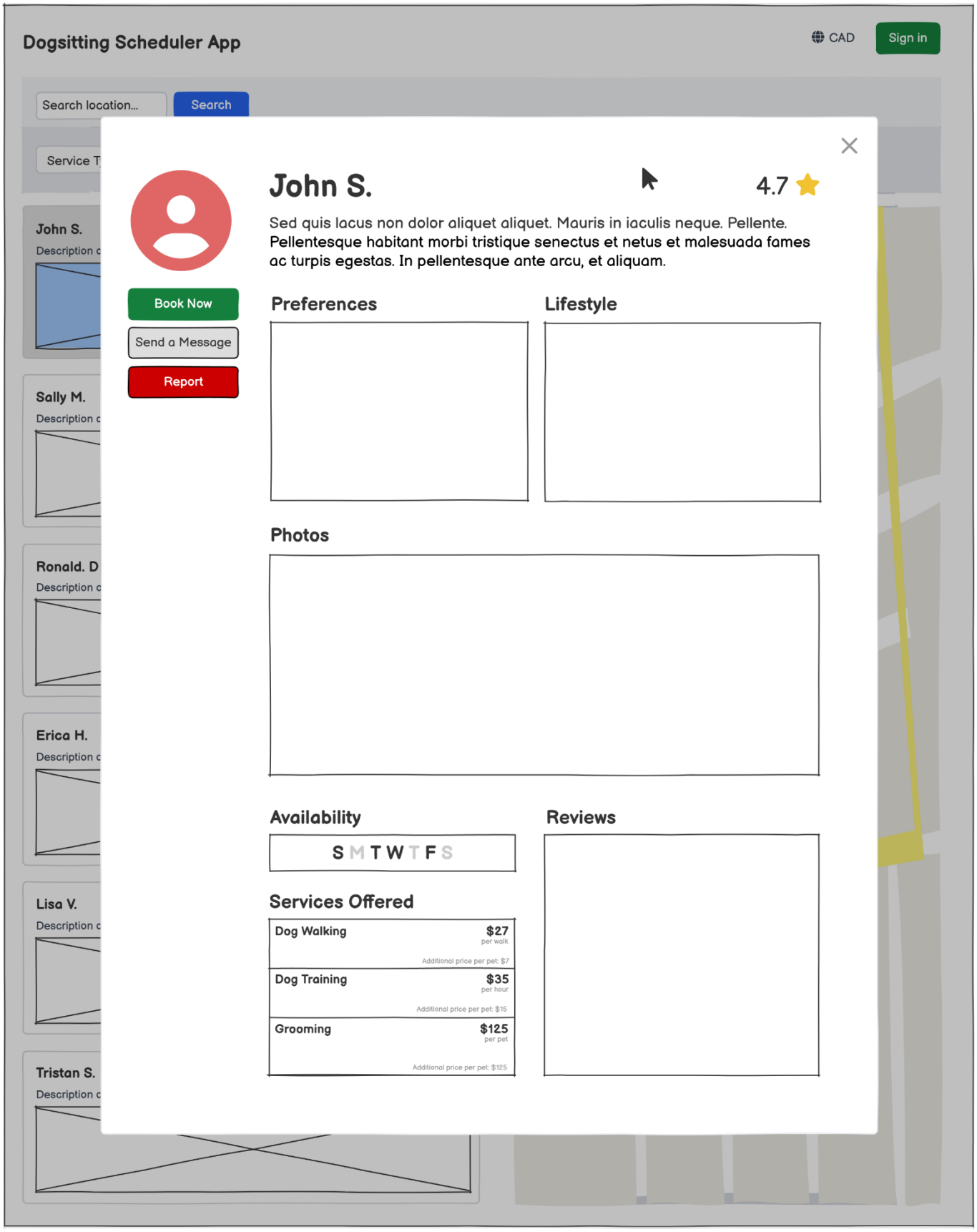
Dog Sitter: Account Registration (Setting Services Offered)



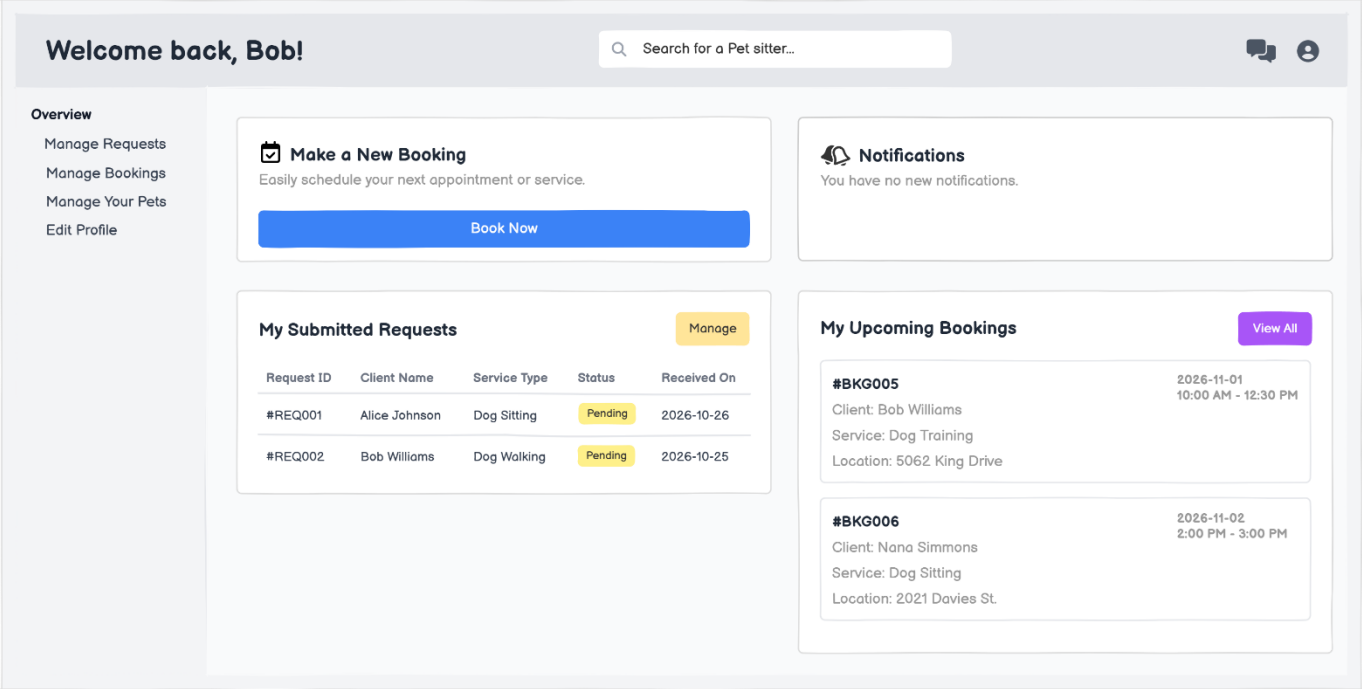
Finding Sitters



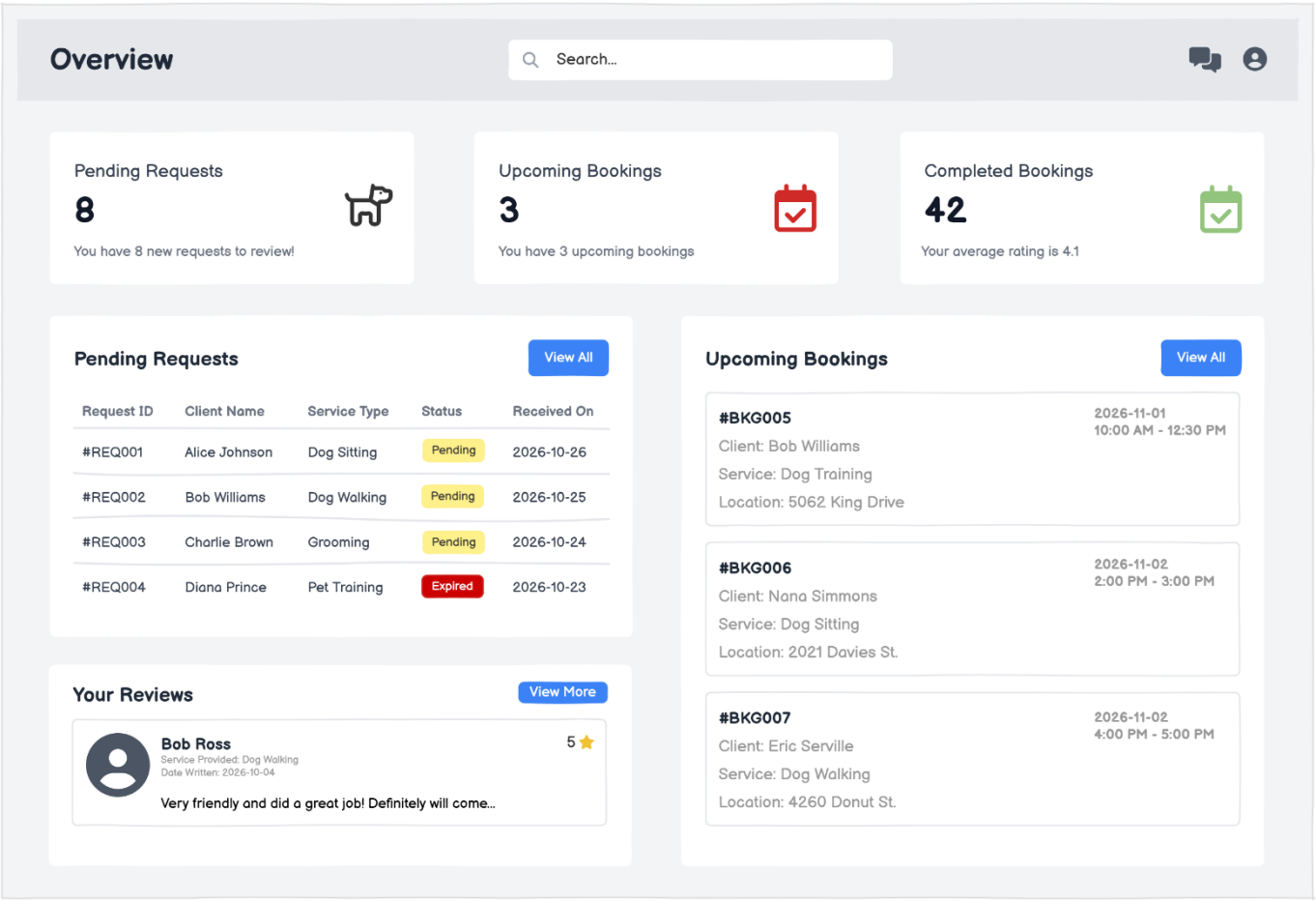
Viewing a Sitter's Profile Page



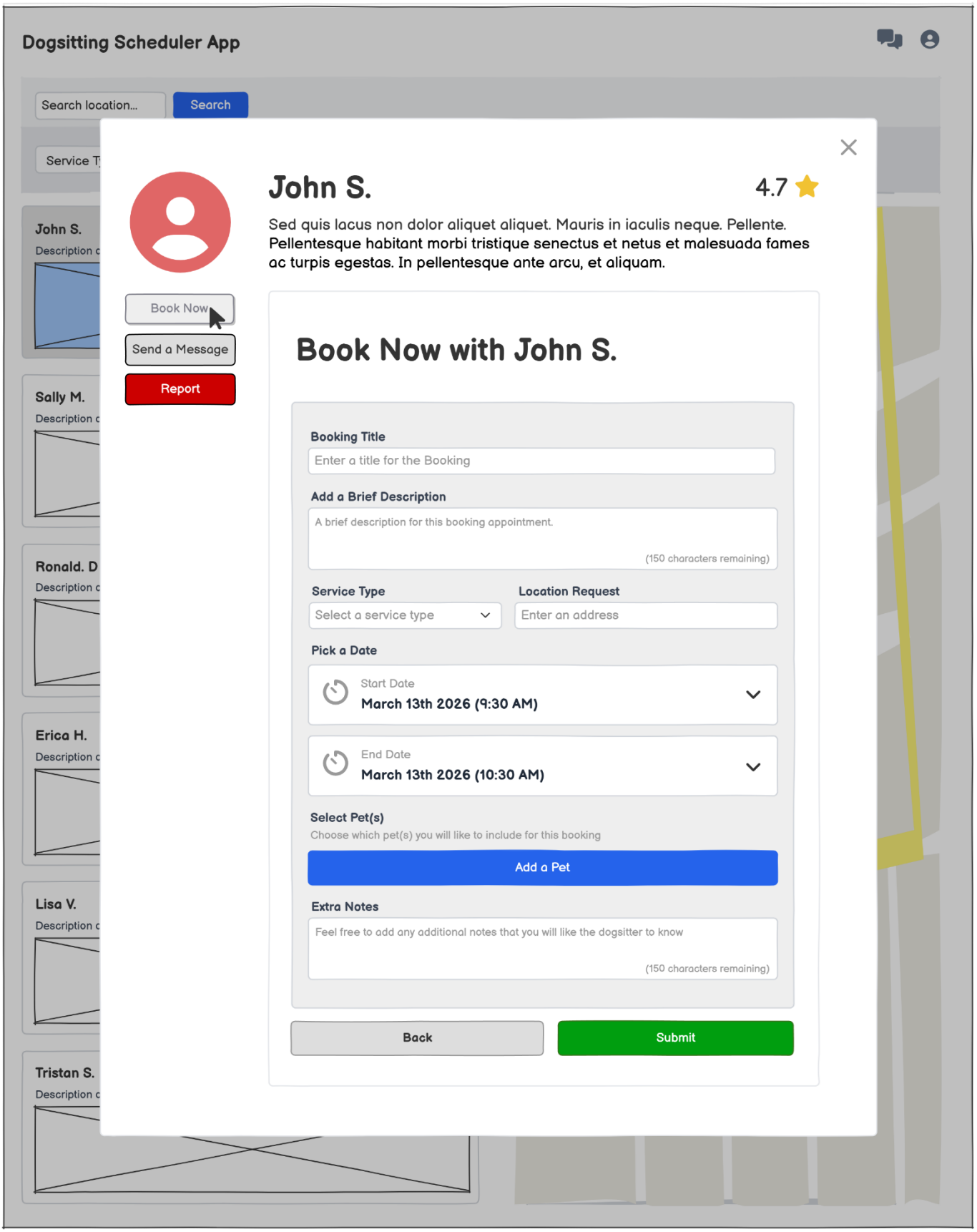
Dog Owner: Dashboard



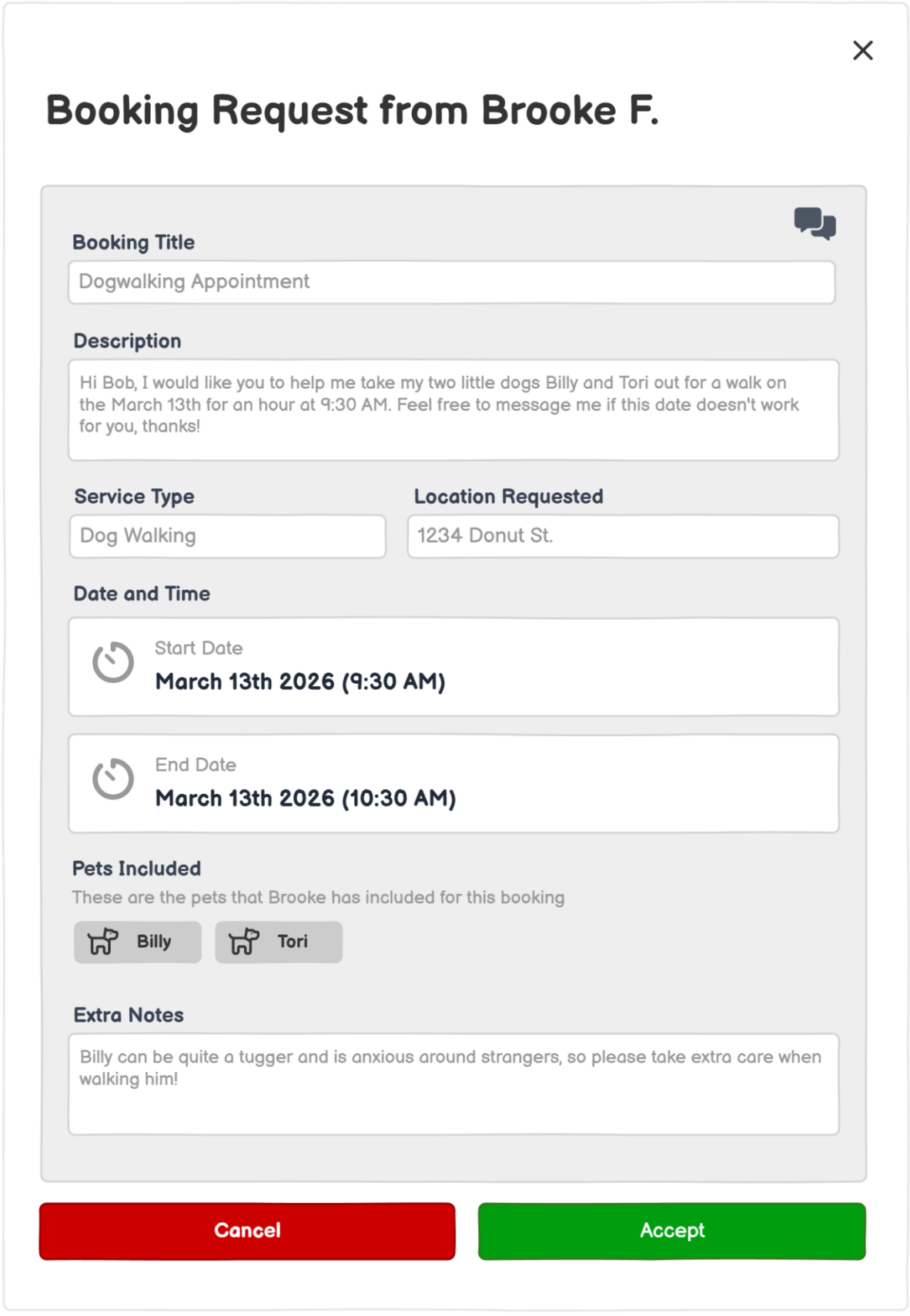
Dog Sitter: Dashboard



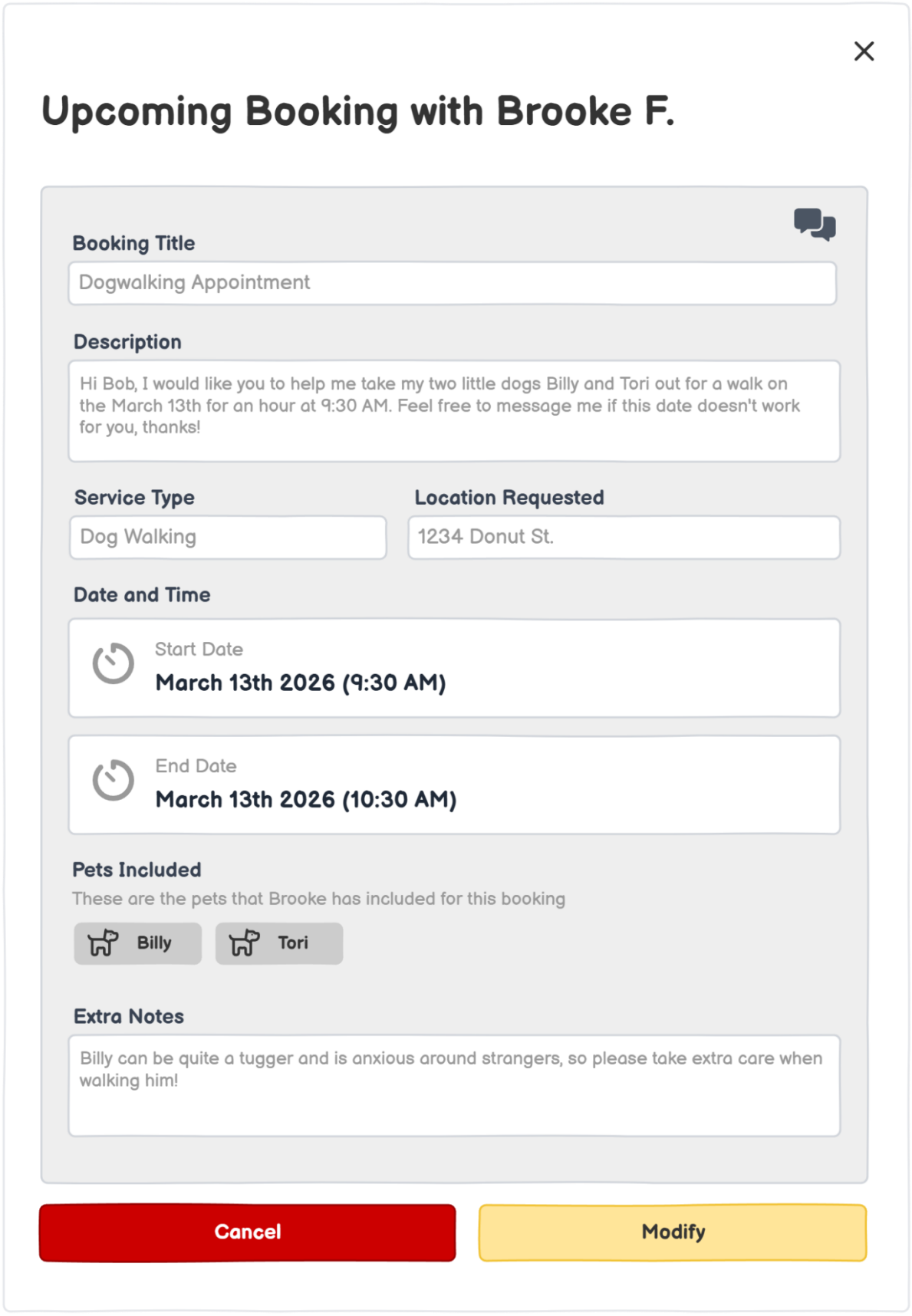
Dog Owner: Creating a Booking Request



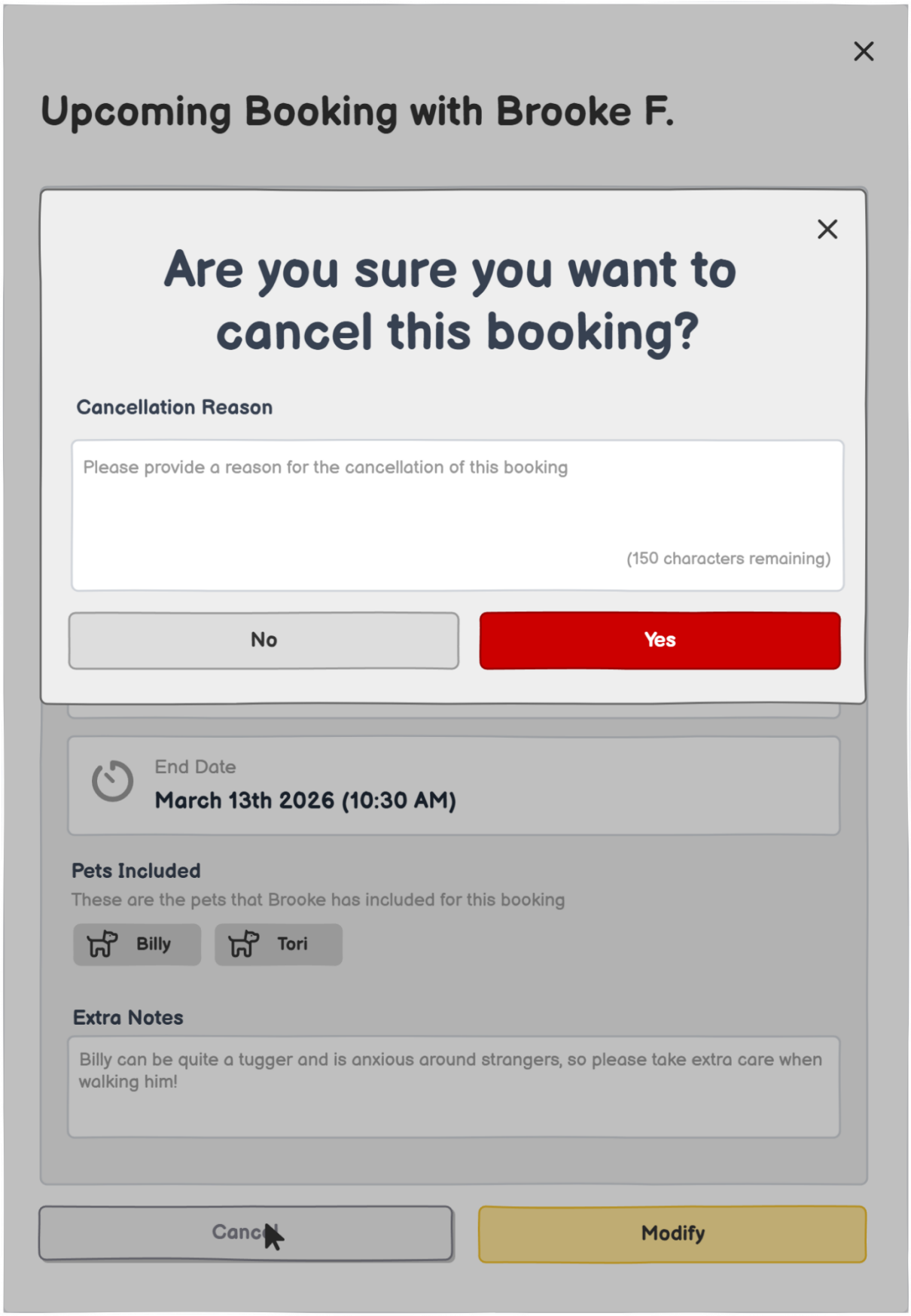
Dog Sitter: Receiving a Booking Request



All Users: Viewing a Booking



All Users: Canceling an Existing Booking



API DESIGN

This section will define some of the API endpoints that will be used by the system and its users. API endpoints allow users (clients) and the system (server) to exchange information with each other through HTTP requests. The information exchanged will be in JSON format.

OVERVIEW

The system will implement a RESTful API architecture to facilitate communication between the frontend application, backend services, and database. API endpoints will allow Dog Owners and Dog Sitters to securely interact with the system by allowing them to perform CRUD operations on entities such as **Requests**, **Bookings**, or **Pets**.

The API will use standard HTTP methods such as *GET, POST, PUT*, and *DELETE* to perform create, retrieve, update, and delete operations on entities defined in **Entity Relationship Diagram (ERD)** section. Most entities defined in the ERD section will have corresponding API endpoints that will allow users to interact with the system.

ENDPOINT DEFINTIONS

The following endpoints below represent API routes that may be implemented within the system. Please note that these endpoints may change and may not be included in the final product.

User Authentication Endpoints

Endpoints	HTTP Request	CRUD Action	Description
/api/auth/register	POST	Create	Creates a new user account
/api/auth/login	POST	Read	Authenticates the user when logging in
/api/auth/logout	POST	Read	Logs the user out (invalidate all granted tokens)
/api/conversations/{con_id}/messages/{sender_id}	GET	Read	Gets all messages from a specific conversation between two users
/api/conversations/{con_id}/messages/{sender_id}	POST	Create	Creates a message for a specific conversation between two users

User Endpoints (CRUD)

NOTE: A **User** can be either a DogOwner or a DogSitter; this is defined in the *UserRole* attribute.

Endpoints	HTTP Request	CRUD Action	Description
/api/users	GET	Read	Get all users
/api/users/{id}	GET	Read	Gets a specific user
/api/users/{id}	PUT	Update	Updates a specific user
/api/users/{id}	DELETE	Delete	Deletes a specific user

Pet Endpoints

Endpoints	HTTP Request	CRUD Action	Description
/api/pets	POST	Create	Creates a specific pet
/api/pets	GET	Read	Get all pets in DB (for testing only)
/api/pets/{pet_id}	GET	Read	Gets a specific pet
/api/pets/{pet_id}	PUT	Update	Updates a specific pet
/api/pets/{pet_id}	DELETE	Delete	Deletes a specific pet
/api/pets/{pet_id}/requests	GET	Read	Get all requests performed on a specific pet
/api/users/{user_id}/pets	GET	Read	Gets all pets belonging to a specific user

Booking Request Endpoints

Endpoints	HTTP Request	CRUD Action	Description
/api/requests	GET	Read	Get all requests in DB (for testing only)
/api/requests	POST	Create	Creates a new request
/api/requests/{id}	GET	Read	Gets a specific request
/api/requests/{id}	PUT	Update	Updates a specific request
/api/users/{user_id}/requests	GET	Read	Get all requests for a specific user

Booking Endpoints

Endpoints	HTTP Request	CRUD Action	Description
/api/bookings	GET	Read	Get all bookings in DB (for testing only)
/api/bookings	POST	Create	Creates a new booking
/api/bookings/{id}	GET	Read	Gets a specific booking
/api/bookings/{id}	PUT	Update	Updates a specific booking
/api/users/{user_id}/bookings	GET	Read	Get all bookings for a specific user

Review Endpoints

Endpoints	HTTP Request	CRUD Action	Description
/api/reviews	GET	Read	Get all reviews in DB (for testing only)
/api/reviews	POST	Create	Creates a new review
/api/reviews/{id}	GET	Read	Gets a specific review
/api/bookings/{id}/review	GET	Read	Gets the review for an associated booking
/api/users/{user_id}/reviews	GET	Read	Gets all reviews written for a specific user

DogSitter Endpoints

Endpoints	HTTP Request	CRUD Action	Description
/api/users/{user_id}/availability	GET	Read	Gets all availabilities for a specific DogSitter
/api/users/{user_id}/services	GET	Read	Gets all services provided for a specific DogSitter
/api/users/{user_id}/preferences	GET	Read	Gets all preferences for a specific DogSitter

Message Endpoints

Endpoints	HTTP Request	CRUD Action	Description
/api/users/{user_id}/conversations	GET	Read	Gets all conversations for a specific user
/api/users/{user_id}/messages/{con_id}	GET	Read	Gets all messages in a conversation for a specific user
/api/users/{user_id}/messages/{con_id}	POST	Create	Creates a message for a specific conversation

SECURITY DESIGN

This section describes how the system will secure user data and API endpoints, protect user accounts, enforce authentication and authorization, and controlling access to system resources. The system will implement measures such as password encryption, secure token-based authentication, input validation, and role-based access control to reduce potential security vulnerabilities and unauthorized data access.

PASSWORD STORAGE AND USER AUTHENTICATION

For password storage, the system will use an email and password-based login mechanism. User passwords will not be stored in plain text; instead, they will be salted and securely hashed using the **BCrypt** hashing algorithm before being stored in the database. This ensures that password storage is "one-way", meaning the original password cannot be derived from its hashed value. As a result, even if the database is compromised, attackers will not be able to recover user passwords in plain text, significantly reducing the risk of accounts being comprised.

For user authentication, the system will use **JSON Web Tokens (JWT).** Upon a successful login and verification of user credentials, the server will generate and issue both an access token and a refresh token to the client. The client will store these tokens locally on their machine. The tokens will contain metadata (claims) such as the user's role, which will be used to support authorization and prevent access to specific API endpoints and server resources. In the event that the metadata in the tokens are tampered with, the signature will be invalidated; thus – forcing the user to re-authenticate themselves.

AUTHORIZATION MODEL

For authorization, the system will enforce **role-based access control** to restrict access to specific API endpoints. Access to protected resources will be determined by the user's assigned role. For example, authenticated users with a DogOwner role will be prevented from accessing or modifying DogSitter-specific data and endpoints. This ensures that users can only perform actions and access resources that are permitted for their role within the system.

TOKEN AND SESSION MANAGEMENT

JWT tokens issued to successfully authenticated users include an expiration time to limit their access time to system resources. When a client presents a valid and unexpired access token, the server will accept the request as authenticated and allow access to the protected resource accordingly. This represents a user session.

As long as the access token remains valid and unaltered, the user is considered to have an active session and may access protected resources without re-authentication. Once the token expires, or if it is invalidated or tampered with, the session is effectively terminated, and the user is required to re-authenticate to obtain a new token.

DATA PROTECTION

For data protection, the system will use **Nginx** as a reverse proxy to handle incoming client connections and terminate HTTPS connections using TLS encryption. TLS certificates will be issued by a certificate authority. Nginx will be configured to fetch, distribute, and present these certificates upon incoming client request to the server. By doing so, the entire connection between clients and the server will always be encrypted, therefore - protecting sensitive data from being intercepted or tampered with.

CHANGE LOG

Date	Description
May 26th, 2026	Design Report (v1) completed
October 30th, 2026	Updated entity relationships (ERD)